BILKUL

GOOGLE SWE 90-DAY INTERVIEW OPERATING SYSTEM

7-Day Pattern Mastery + LLD + HLD + Interview Simulation

From Pattern Recognition → Problem Solving → System Design → Interview Ready

64 Patterns | Java Templates | LLD + HLD | Resume Deep Dive | Mock Interviews

Target: Google SWE L4 (L5 stretch) · Java Backend · 8-10 hrs/day × 7 days first pass

Optimized for active learning, recall, recognition — not passive reading.

Interview Operating System — Navigation

Click to jump. Checkboxes save progress in browser (localStorage).

Core Philosophy

> Why compress DSA into ~50 patterns? Because interviews test recognition + reasoning, not memorizing 300 solutions.

Your goal in 7 days: Build a mental framework to answer:

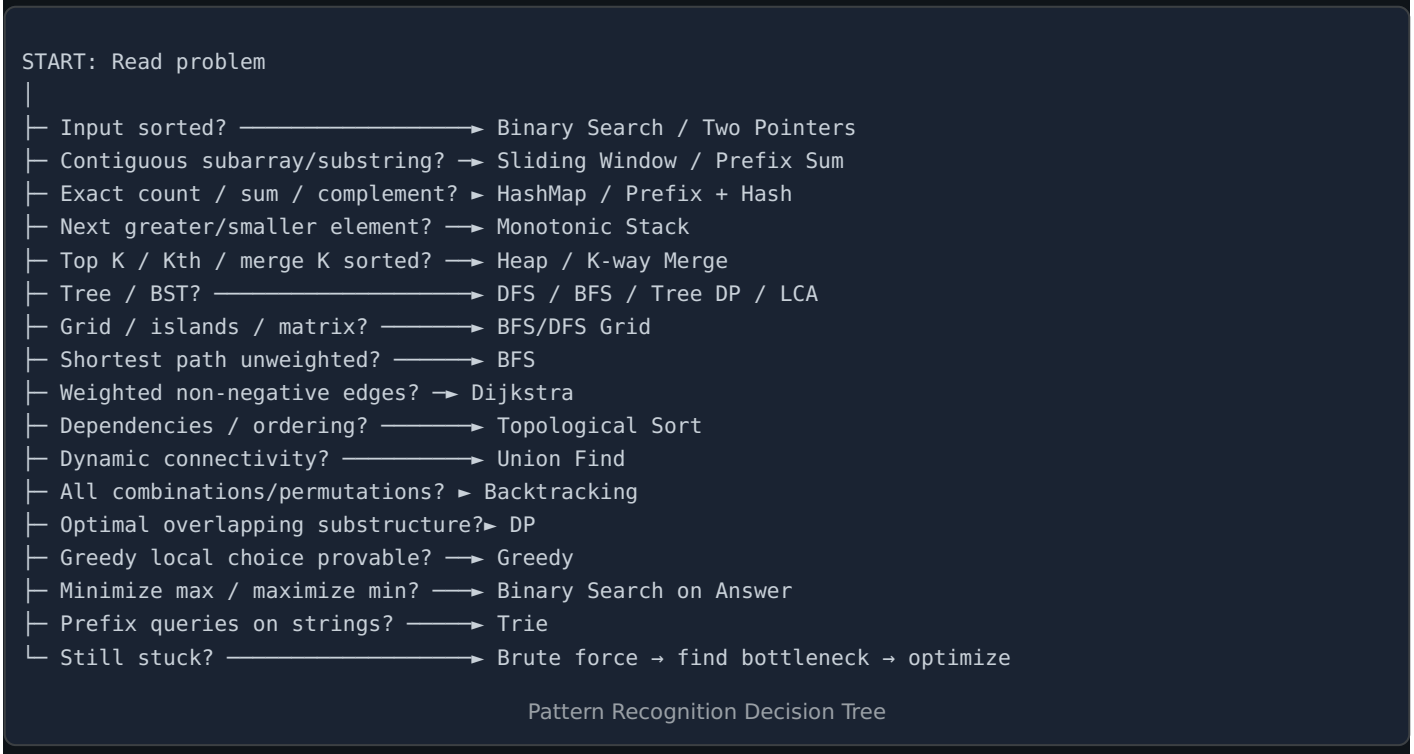
1. What pattern is this?
2. Why does that pattern apply?
3. What is the brute-force solution?
4. What observation eliminates brute force?
5. What data structure/algorithm should I use?
6. What is optimal time and space complexity?
7. What edge cases can break my solution?
8. How would I explain this in a Google interview?
9. How would I extend if constraints change?

These are reusable frameworks — some problems combine multiple patterns. Deep understanding beats excessive content.

L4 vs L5 Expectations

L4	L5
Recognize patterns, optimal solutions, clean Java	Trade-off analysis, scale 10x, org-level decisions
Complete LLD with SOLID	Multi-tenant, observability, migration strategy
End-to-end HLD with estimation	Multi-region consistency, cost, failure domains

Pattern Recognition Engine



Pattern Recognition Decision Tree

30-Second DSA Diagnosis Framework

For every new problem, answer these in order:

1. **Input?** Array, string, tree, graph, matrix, stream?
2. **Output?** Value, index, count, list, boolean?
3. **Ordering important?** Yes → sort/pointers; No → hash/set
4. **Sorted?** Yes → BS/two pointers
5. **Contiguous?** Yes → sliding window/prefix
6. **Existence / count / min / max?** Maps to hash, BS, heap, DP
7. **All possibilities?** Backtracking
8. **Repeated substructure?** DP
9. **Monotonic property?** BS on answer, two pointers, monotonic stack
10. **Constraints?** $n \leq 20$ → bitmask; $n \leq 10^5$ → O(n) or O(n log n)

Answer Pattern	Likely Techniques
Sorted + find	Binary Search, Two Pointers
Contiguous + constraint	Sliding Window, Prefix Sum
Count / frequency	HashMap, Prefix + Hash
Top K	Heap, QuickSelect
Tree path/subtree	DFS, Tree DP
Shortest unweighted	BFS
Dependencies	Topological Sort, DFS cycle
All subsets/perms	Backtracking
Min-max optimization	DP, Greedy, BS on Answer

Quiz: Array is sorted, find two numbers with given sum. First pattern?

Reveal Answer

Quiz: Find shortest path in unweighted grid with obstacles.

Reveal Answer

1-Page: 30-Second Diagnosis

- Sorted? → BS / Two Ptr
- Contiguous? → Window / Prefix
- Count? → HashMap
- Top K? → Heap
- Next greater? → Mono Stack
- Tree? → DFS/BFS
- Grid? → BFS/DFS 4-dir
- Shortest unweighted? → BFS
- Weighted? → Dijkstra
- Deps? → Topo Sort
- Components? → UF/DFS
- All combos? → Backtrack
- Optimal substructure? → DP
- Min max answer? → BS on Answer

DSA Pattern Cards

64 reusable frameworks — not 300 random problems. Some problems combine multiple patterns.

ARRAY / HASHING

#1 ARRAY / HASHING Mastered (+10 XP)

Frequency Map

Why does Frequency Map work? Count occurrences to answer existence, uniqueness, or anagram questions in O(n).

Mental Model: A tally board — each element gets a counter; compare or aggregate counts.

Recognition Signals: "count", "frequency", "anagram", "duplicate", "most common", "two sum complement"

When to use: Need counts, complements, or character frequency comparisons.

When NOT to use: Need ordering of indices or contiguous subarray properties only.

Brute Force

Nested loops counting each element $O(n^2)$.

Optimal Approach

Single pass: map value → count or last index.

Key Observation

If answer depends on how many times X appears, one pass with HashMap suffices.

Complexity

Time: $O(n)$ · Space: $O(n)$

Algorithm

- Initialize HashMap
- Iterate input
- Update counts or check complement
- Return result

Java Template

```
Map<Integer, Integer> freq = new HashMap<>();
for (int x : nums) freq.merge(x, 1, Integer::sum);
```

Common Mistakes

- Forgetting Integer overflow on sum
- Using HashMap when TreeMap order needed

Edge Cases

- Empty array
- All same elements
- Integer.MIN_VALUE keys

Easy Examples

- **Two Sum** (Easy): Find indices with target sum — Store complement in map

- **Valid Anagram** (Easy): Compare char frequencies — Single freq map or array[26]

Medium Examples

- **Top K Frequent** (Medium): K most frequent elements — Freq map + min-heap of size k
- **Group Anagrams** (Medium): Group strings by signature — Map sorted key → list

Hard / Interview Examples

- **Subarray Sum Equals K** (Hard): Count subarrays summing to k — Prefix sum + freq map of prefix counts

30-Second Recognition: See 'count' or 'how many times' → think HashMap.

> Trap: Using HashMap when you need sorted order — use TreeMap.

> Interview: Follow-up: What if input doesn't fit in memory? · Variation: Change to exact k distinct characters → sliding window + freq map.

#2 ARRAY / HASHING ☐ Mastered (+10 XP)

Prefix Sum

Why does Prefix Sum work? Precompute cumulative sums to answer range-sum queries in O(1).

Mental Model: Running total at each index — range [i,j] = prefix[j+1] - prefix[i].

Recognition Signals: "subarray sum", "range sum", "continuous", "equilibrium", "modulo"

When to use: Multiple range-sum queries or subarray sum equals target via prefix difference.

When NOT to use: Single pass max/min without range structure.

Brute Force

Recompute sum for every range O(n²).

Optimal Approach

Build prefix[] in one pass; use map prefix→count for counting problems.

Key Observation

sum(i..j) = prefix[j+1]-prefix[i]; store prefix counts for exact-k subarrays.

Complexity

Time: `O(n) build, O(1) query` · Space: `O(n)`

Algorithm

- prefix[0]=0
- For each i: prefix[i+1]=prefix[i]+nums[i]
- Query or update map

Java Template

```
int[] prefix = new int[n+1];
for (int i=0;i<n;i++) prefix[i+1]=prefix[i]+nums[i];
```

Common Mistakes

- Off-by-one in prefix indexing
- Not handling negative mod

Edge Cases

- Empty prefix
- All negatives
- Modulo with negative remainder

Easy Examples

- **Range Sum Query** (Easy): Sum between i and j — prefix[j+1]-prefix[i]
- **Equilibrium Index** (Easy): Index where left sum = right sum — Total sum - left == left

Medium Examples

- **Subarray Sum Divisible by K** (Medium): Count subarrays — Prefix mod k + freq map

Hard / Interview Examples

- **Max Subarray with at most K distinct** (Hard): At most K distinct chars — Sliding window + freq map

30-Second Recognition: Contiguous + sum/count → prefix sum.

> Trap: Forgetting mod normalization for negative numbers.

> Interview: Follow-up: How to handle updates + queries? → Fenwick/segment tree. · Variation: 2D prefix sum for matrix region queries.

#3 ARRAY / HASHING Mastered (+10 XP)

Difference Array

Why does Difference Array work? Apply range updates in O(1) per update, reconstruct final array at end.

Mental Model: Instead of adding v to [l,r] repeatedly, mark +v at l and -v at r+1.

Recognition Signals: "range update", "multiple increments", "flight bookings", "interval add"

When to use: Many range increment operations on array, then final state needed.

When NOT to use: Few point updates or need intermediate states after each query.

Brute Force

Loop l..r for each update O(n*q).

Optimal Approach

diff[l]+=v; diff[r+1]-=v; prefix sum diff → result.

Key Observation

Difference array converts range adds to two boundary updates.

Complexity

Time: $O(n+q)$ · Space: $O(n)$

Algorithm

- Initialize diff[n+1]
- For each [l,r,v]: diff[l]+=v; diff[r+1]-=v
- Prefix sum diff into ans

Java Template

```
int[] diff = new int[n+1];
diff[l] += val; diff[r+1] -= val;
// after all updates: prefix sum
```

Common Mistakes

- r+1 out of bounds
- Confusing with prefix sum

Edge Cases

- l=0
- r=n-1
- Overlapping ranges

Easy Examples

- **Range Addition** (Easy): Add v to [l,r] k times — Difference array

Medium Examples

- **Corporate Flight Bookings** (Medium): LeetCode 1109 — Classic diff array

30-Second Recognition: Multiple range adds → difference array.

> Trap: Using when only one range query needed.

> Interview: Follow-up: 2D difference array for matrix? · Variation: Combine with lazy propagation in segment trees.

#4 ARRAY / HASHING ☐ Mastered (+10 XP)

Two Pointers

· Why does Two Pointers work? Two indices moving toward each other or same direction to reduce O(n²) to O(n).

Mental Model: Sorted array: left/right squeeze; unsorted: slow/fast or same-direction chase.

Recognition Signals: "sorted", "pair", "palindrome", "remove duplicates", "container"

When to use: Sorted input or pair/triplet search; in-place compaction.

When NOT to use: Unsorted with no monotonic structure and need all subsets.

Brute Force

Check all pairs O(n²).

Optimal Approach

l=0, r=n-1; move pointer that improves target condition.

Key Observation

Sorted order lets you decide which pointer to move based on sum comparison.

Complexity

Time: `O(n) after sort O(n log n)` · Space: `O(1) extra`

Algorithm

- Sort if needed
- Initialize l,r
- While l

Java Template

```
int l = 0, r = nums.length - 1;
while (l < r) {
    // move l++ or r-- based on condition
}
```

Common Mistakes

- Not sorting when required
- Infinite loop on equal moves

Edge Cases

- All elements same
- Two elements only
- Duplicates in 3-sum

Easy Examples

- **Two Sum II** (Easy): Sorted two sum — Opposite pointers
- **Valid Palindrome** (Easy): Skip non-alnum, compare ends — Two pointers from both ends

Medium Examples

- **3Sum** (Medium): Fix one + two pointer — Sort + for i: two ptr on rest
- **Container With Most Water** (Medium): Move shorter side — Greedy two pointer

Hard / Interview Examples

- **Trapping Rain Water** (Hard): Two pointer with maxL/maxR — Track left/right max heights

30-Second Recognition: Sorted + pair → two pointers.

> Trap: Using on unsorted without clear move rule.

> Interview: Follow-up: Why move shorter side in container problem? · Variation: 4Sum → reduce to 3Sum + hash or two pointers.

#5 ARRAY / HASHING Mastered (+10 XP)

Fast & Slow Pointers

Why does Fast & Slow Pointers work? Two pointers at different speeds detect cycles or find middle.

Mental Model: Tortoise and hare — if cycle exists, fast eventually meets slow.

Recognition Signals: "cycle", "middle", "linked list", "duplicate number"

When to use: Cycle detection, find middle, find cycle start.

When NOT to use: Array without implicit next pointer structure (unless index as pointer).

Brute Force

HashSet to detect revisit O(n) space.

Optimal Approach

slow+=1, fast+=2 per iteration until meet or fast hits end.

Key Observation

In cycle, fast gains 1 step per lap on slow → must meet.

Complexity

Time: O(n) · Space: O(1)

Algorithm

- Initialize slow=fast=head
- While fast and fast.next: advance
- Check meeting point

Java Template

```
ListNode slow=head, fast=head;
while (fast!=null && fast.next!=null) {
  slow=slow.next; fast=fast.next.next;
}
```

Common Mistakes

- Not checking fast.next for null
- Wrong start for cycle entry

Edge Cases

- No cycle
- Single node
- Cycle at head

Easy Examples

- **Middle of List** (Easy): Return middle node — Fast 2x slow

Medium Examples

- **Linked List Cycle** (Medium): Detect cycle — Floyd's algorithm

Hard / Interview Examples

- **Find Duplicate Number** (Hard): Array as linked list — Floyd cycle detection on index chain

30-Second Recognition: Cycle or middle in linked structure → fast/slow.

> Trap: Applying without understanding implicit graph.

> Interview: Follow-up: Find cycle start node — why reset slow to head? · Variation: Happy number — same cycle detection on digit square sum.

#6 ARRAY / HASHING Mastered (+10 XP)

Sliding Window — Fixed

Why does Sliding Window — Fixed work? Window of fixed size k slides across array.

Mental Model: Sliding frame of width k — add right, remove left.

Recognition Signals: "window size k", "subarray of length", "maximum of all windows"

When to use: Fixed-length contiguous subarray/substring analysis.

When NOT to use: Variable constraint on window content.

Brute Force

Recompute each window from scratch.

Optimal Approach

Maintain window state; for each r, if r>=k update with nums[r-k].

Key Observation

Only one element enters/exits per slide — update in O(1).

Complexity

Time: $O(n)$ · Space: $O(1)$

Algorithm

- Compute first window
- Slide: add nums[r], remove nums[r-k]
- Track max/min

Java Template

```
int l = 0;
for (int r = 0; r < n; r++) {
  // expand window with r
  while (r - l + 1 > k) { l++ }
  // update answer
}
```

Common Mistakes

- Off-by-one on window size
- Not handling k > n

Edge Cases

- k=1
- k=n
- k > n

Easy Examples

- **Sliding Window — Fixed** (Easy): Easy variant — Maintain window state; for each r, if r>=k update with nums[r-k].

Medium Examples

- **Sliding Window — Fixed** (Medium): Medium variant — Maintain window state; for each r, if r>=k update with nums[r-k].

30-Second Recognition: Fixed k in problem → fixed sliding window.

> Trap: Using fixed window when constraint is 'at most/at least'.

> Interview: Follow-up: Deque for max in sliding window? · Variation: Variable window instead when constraint is on content not length.

#7 ARRAY / HASHING Mastered (+10 XP)

Sliding Window — Variable

Why does Sliding Window — Variable work? Expand right until invalid, shrink left until valid — optimal contiguous subarray.

Mental Model: Rubber band — stretch until rule breaks, then contract.

Recognition Signals: "longest substring", "at most k distinct", "minimum window", "subarray sum >= target"

When to use: Contiguous subarray with constraint on content (distinct chars, sum threshold).

When NOT to use: Non-contiguous selection or fixed k only.

Brute Force

Check all subarrays O(n²).

Optimal Approach

Expand r, while invalid shrink l, track best.

Key Observation

Once window invalid, only shrinking left can restore validity — monotonic l.

Complexity

Time: `O(n)` · Space: `O(k) for map`

Algorithm

- l=0
- Expand r updating state
- While invalid: shrink l
- Update answer

Java Template

```
int l = 0;
for (int r = 0; r < n; r++) {
  // expand window with r
  while (/* invalid */) { /* shrink with l++ */ }
  // update answer
}
```

Common Mistakes

- Not shrinking enough
- Updating answer at wrong time

Edge Cases

- Empty string
- All same char
- No valid window

Easy Examples

- **Sliding Window — Variable** (Easy): Easy variant — Expand r, while invalid shrink l, track best.

Medium Examples

- **Sliding Window — Variable** (Medium): Medium variant — Expand r, while invalid shrink l, track best.

30-Second Recognition: Substring/subarray + 'longest/shortest/at most' → variable window.

> Trap: Forgetting to update answer after shrink loop.

> Interview: Follow-up: Minimum window substring follow-up? · Variation: At most K vs exactly K — adjust with at_most(K) - at_most(K-1).

#8 ARRAY / HASHING Mastered (+10 XP)

Kadane / Running-State Optimization

Why does Kadane / Running-State Optimization work? Track best ending-here subarray by resetting when current sum goes negative.

Mental Model: At each step: extend current segment or start fresh.

Recognition Signals: "maximum subarray", "maximum product", "best sum ending here"

When to use: Max/min subarray sum or product with local reset rule.

When NOT to use: Need count of subarrays or non-contiguous pick.

Brute Force

All subarrays O(n²).

Optimal Approach

cur = max(nums[i], cur+nums[i]); best = max(best, cur).

Key Observation

Optimal subarray ending at i either extends i-1 or starts at i.

Complexity

Time: O(n) · Space: O(1)

Algorithm

- cur=nums[0], best=nums[0]
- For i=1..n: cur=max(nums[i], cur+nums[i])
- best=max(best,cur)

Java Template

```
int cur=nums[0], best=nums[0];
for(int i=1;i<n;i++){cur=Math.max(nums[i],cur+nums[i]);best=Math.max(best,cur);}
```

Common Mistakes

- All negative array
- Product kadane needs min tracking

Edge Cases

- All negatives
- Single element

Easy Examples

- **Kadane / Running-State Optimization** (Easy): Easy variant — cur = max(nums[i], cur+nums[i]); best = max(best, cur).

Medium Examples

- **Kadane / Running-State Optimization** (Medium): Medium variant — cur = max(nums[i], cur+nums[i]); best = max(best, cur).

30-Second Recognition: Max subarray sum → Kadane.

> Trap: Not handling all-negative (answer is max element).

> Interview: Follow-up: Circular array max subarray? · Variation: Max product subarray — track max and min (negative flip).

#9 ARRAY / HASHING Mastered (+10 XP)

Sorting + Greedy

Why does Sorting + Greedy work? Sort to expose structure, then greedy choice becomes obvious.

Mental Model: Put elements in order so local optimal = global optimal.

Recognition Signals: "minimum arrows", "intervals", "assign cookies", "largest number"

When to use: Greedy proof relies on sorted order.

When NOT to use: DP needed with overlapping subproblems.

Brute Force

Try all permutations.

Optimal Approach

Sort by key; one pass greedy assignment.

Key Observation

After sort, only compare neighbors or sweep line order matters.

Complexity

Time: $O(n \log n)$ · Space: $O(1)$

Algorithm

- Sort
- Greedy scan
- Update answer

Java Template

```
Arrays.sort(intervals, (a,b)->a[0]-b[0]);
```

Common Mistakes

- Wrong comparator
- Integer overflow in compare

Edge Cases

- Equal elements
- Already sorted

Easy Examples

- **Sorting + Greedy** (Easy): Easy variant — Sort by key; one pass greedy assignment.

Medium Examples

- **Sorting + Greedy** (Medium): Medium variant — Sort by key; one pass greedy assignment.

30-Second Recognition: Sort mentioned or intervals → sort + greedy.

Trap: Greedy without proving exchange argument.

Interview: Follow-up: Prove why greedy works? · Variation: Custom comparator for largest number concatenation.

#10 ARRAY / HASHING ☐ Mastered (+10 XP)

Intervals / Merge Intervals

Why does Intervals / Merge Intervals work? Sort by start, merge overlapping intervals.

Mental Model: Stack of merged ranges — extend end if overlap.

Recognition Signals: "merge intervals", "meeting rooms", "overlapping", "insert interval"

When to use: Interval scheduling, overlap detection, room count.

When NOT to use: Points not intervals.

Brute Force

Compare all pairs $O(n^2)$.

Optimal Approach

If cur.start <= last.end merge else append.

Key Observation

After sort by start, only need to compare with last merged.

Complexity

Time: $O(n \log n)$ · Space: $O(n)$

Algorithm

- Sort by start
- Init list with first
- For each: merge or add

Java Template

```
Arrays.sort(intervals,(a,b)->a[0]-b[0]);
List<int[]> res=new ArrayList<>();
```

Common Mistakes

- Not sorting
- Off-by-one on overlap

Edge Cases

- No overlap
- All overlap
- Single interval

Easy Examples

- **Intervals / Merge Intervals** (Easy): Easy variant — If cur.start <= last.end merge else append.

Medium Examples

- **Intervals / Merge Intervals** (Medium): Medium variant — If cur.start <= last.end merge else append.

30-Second Recognition: Intervals → sort + merge or sweep.

> Trap: Confusing [1,4] and [4,5] as overlapping (problem dependent).

> Interview: Follow-up: Minimum meeting rooms? · Variation: Sweep line with start +1 end -1 events.

BINARY SEARCH

#11 BINARY SEARCH Mastered (+10 XP)

Classic Binary Search

Why does Classic Binary Search work? Halve search space on sorted array for target.

Mental Model: Phone book — eliminate half each guess.

Recognition Signals: "sorted array", "find target", "first occurrence", "search insert"

When to use: Sorted data, find index/value.

When NOT to use: Unsorted or need all solutions via brute.

Brute Force

Linear scan O(n).

Optimal Approach

lo, hi inclusive; mid = lo + (hi-lo)/2.

Key Observation

Monotonic predicate: if a[mid] too big, answer in left half.

Complexity

Time: $O(\log n)$ · Space: $O(1)$

Algorithm

- lo=0, hi=n-1
- While lo<=hi compute mid
- Adjust lo/hi

Java Template

```
int lo = 0, hi = n - 1;
while (lo <= hi) {
  int mid = lo + (hi - lo) / 2;
  if (predicate(mid)) hi = mid - 1; else lo = mid + 1;
}
```

Common Mistakes

- hi=mid instead of mid-1
- Overflow on (lo+hi)/2

Edge Cases

- Empty
- Single element
- Target not present

Easy Examples

- **Classic Binary Search** (Easy): Easy variant — lo, hi inclusive; mid = lo + (hi-lo)/2.

Medium Examples

- **Classic Binary Search** (Medium): Medium variant — lo, hi inclusive; mid = lo + (hi-lo)/2.

30-Second Recognition: Sorted + find → binary search.

> Trap: Using BS on unsorted array.

> Interview: Follow-up: Find first/last occurrence? · Variation: Lower bound template.

#12 BINARY SEARCH Mastered (+10 XP)

Binary Search on Answer

Why does Binary Search on Answer work? Search answer space [min,max] where feasibility is monotonic.

Mental Model: Not searching index — searching the answer value itself.

Recognition Signals: "minimum maximum", "maximize minimum", "feasible?", "k partitions"

When to use: Minimize max or maximize min with monotonic feasibility check.

When NOT to use: No monotonic can/can't predicate on answer.

Brute Force

Try all answers O(n * range).

Optimal Approach

BS on range; check(mid) greedy simulation.

Key Observation

If x feasible then x+1 feasible (or reverse) → BS on answer.

Complexity

Time: $O(n \log R)$ · Space: $O(1)$

Algorithm

- Find lo, hi bounds
- While lo
- check(mid) adjust

Java Template

```
int lo=min, hi=max;
while(lo<hi){
  int mid=lo+(hi-lo)/2;
  if(can(mid)) hi=mid; else lo=mid+1;
}
```

Common Mistakes

- Wrong boundary (lo
- Non-monotonic check

Edge Cases

- Answer at boundary
- All same values

Easy Examples

- **Binary Search on Answer** (Easy): Easy variant — BS on range; check(mid) greedy simulation.

Medium Examples

- **Binary Search on Answer** (Medium): Medium variant — BS on range; check(mid) greedy simulation.

30-Second Recognition: Minimize the maximum / maximize the minimum → BS on answer.

> Trap: Binary searching when predicate isn't monotonic.

> Interview: Follow-up: Prove monotonicity? · Variation: Split array largest sum — BS on answer + greedy partition count.

#13 BINARY SEARCH Mastered (+10 XP)

Lower Bound / Upper Bound

Why does Lower Bound / Upper Bound work? Find first index where nums[i] >= target (lower) or > target (upper).

Mental Model: BS variant for insert position or count of target.

Recognition Signals: "insert position", "count occurrences", "first >= x"

When to use: Need first/last position or count in sorted array.

When NOT to use: Unsorted frequency — use HashMap.

Brute Force

Two linear scans.

Optimal Approach

BS with lo<hi, hi=mid when nums[mid]>=target.

Key Observation

lower_bound: first idx with val >= target.

Complexity

Time: $O(\log n)$ · Space: $O(1)$

Algorithm

- lo=0, hi=n
- While lo
- Return lo

Java Template

```
while(lo<hi){int mid=lo+(hi-lo)/2;if(nums[mid]<target)lo=mid+1;else hi=mid;}
```

Common Mistakes

- Confusing lower vs upper
- Empty array hi=n

Edge Cases

- All less than target
- All equal to target

Easy Examples

- **Lower Bound / Upper Bound** (Easy): Easy variant — BS with lo<hi, hi=mid when nums[mid]>=target.

Medium Examples

- **Lower Bound / Upper Bound** (Medium): Medium variant — BS with lo<hi, hi=mid when nums[mid]>=target.

30-Second Recognition: Sorted + insert position → lower bound.

> Trap: Using lo<=hi template incorrectly.

> Interview: Follow-up: Count of target in sorted array? · Variation: upper_bound - lower_bound.

#14 BINARY SEARCH Mastered (+10 XP)

Rotated Sorted Array

Why does Rotated Sorted Array work? One half is always sorted — determine which and BS.

Mental Model: Broken circle — one sorted segment always identifiable at mid.

Recognition Signals: "rotated", "pivot", "find minimum", "search rotated"

When to use: Sorted array rotated at unknown pivot.

When NOT to use: Fully unsorted.

Brute Force

Find pivot then BS O(n).

Optimal Approach

If target in sorted half BS there else other half.

Key Observation

Compare nums[mid] with nums[hi] to know which half sorted.

Complexity

Time: $O(\log n)$ · Space: $O(1)$

Algorithm

- lo, hi
- mid
- Identify sorted half
- BS in correct half

Java Template

```
if(nums[mid]>=nums[lo]){/* left sorted */}else{/* right sorted */}
```

Common Mistakes

- Duplicates break uniqueness
- Wrong half chosen

Edge Cases

- No rotation
- Full rotation
- Duplicates

Easy Examples

- **Rotated Sorted Array** (Easy): Easy variant — If target in sorted half BS there else other half.

Medium Examples

- **Rotated Sorted Array** (Medium): Medium variant — If target in sorted half BS there else other half.

30-Second Recognition: Rotated sorted → modified BS.

> Trap: Duplicates — may degrade to O(n).

> Interview: Follow-up: Find minimum in rotated array? · Variation: With duplicates use while lo

#15 BINARY SEARCH Mastered (+10 XP)

Search on Monotonic Predicate

Why does Search on Monotonic Predicate work? Abstract BS: first x where predicate(x) is true.

Mental Model: Any problem where false,false,...,true,true.

Recognition Signals: "smallest feasible", "first true", "monotonic"

When to use: Custom predicate over integer range.

When NOT to use: Multiple disjoint feasible regions.

Brute Force

Linear scan predicate.

Optimal Approach

Template: find first true.

Key Observation

Predicate flips once → BS.

Complexity

Time: $O(\log R * \text{cost(check)})$ · Space: $O(1)$

Algorithm

- Define check(x)
- BS on domain
- Return lo

Java Template

```
int lo = 0, hi = n - 1;
while (lo <= hi) {
  int mid = lo + (hi - lo) / 2;
  if (check(mid)) hi = mid - 1; else lo = mid + 1;
}
```

Common Mistakes

- Predicate not monotonic
- Infinite loop

Edge Cases

- All false
- All true

Easy Examples

- **Search on Monotonic Predicate** (Easy): Easy variant — Template: find first true.

Medium Examples

- **Search on Monotonic Predicate** (Medium): Medium variant — Template: find first true.

30-Second Recognition: Feasibility flips once → monotonic BS.

> Trap: Assuming monotonicity without verification.

> Interview: Follow-up: How to test check() efficiently? · Variation: Real-valued BS with epsilon for geometry problems.

LINKED LIST

#16 LINKED LIST Mastered (+10 XP)

Reverse Linked List

Why does Reverse Linked List work? Iterative three-pointer reversal in O(n).

Mental Model: prev/curr/next walk reversing links.

Recognition Signals: "reverse", "linked list"

When to use: Reverse in-place singly linked list.

When NOT to use: Need copy not reverse.

Brute Force

Store nodes in stack.

Optimal Approach

prev=null; curr=head; swap links.

Key Observation

Save next before rewiring.

Complexity

Time: O(n) · Space: O(1)

Algorithm

- prev,curr,next
- While curr: reverse link
- Return prev

Java Template

```
ListNode prev=null,cur=head;
while(cur!=null){ListNode n=cur.next;cur.next=prev;prev=cur;cur=n;}
return prev;
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Reverse Linked List Easy** (Easy): Canonical easy problem — prev=null; curr=head; swap links.

Medium Examples

- **Reverse Linked List Medium** (Medium): Interview variant — prev=null; curr=head; swap links.

Hard / Interview Examples

- **Reverse Linked List Hard** (Hard): Google-style twist — prev=null; curr=head; swap links.

30-Second Recognition: Triggers: reverse, linked list

> Trap: Using Reverse Linked List when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#17 LINKED LIST Mastered (+10 XP)

Fast/Slow Pointer (List)

Why does Fast/Slow Pointer (List) work? Cycle and middle detection.

Mental Model: Tortoise-hare on list.

Recognition Signals: "cycle", "middle"

When to use: Cycle or midpoint.

When NOT to use: Array two-pointer without links.

Brute Force

HashSet.

Optimal Approach

Floyd algorithm.

Key Observation

Meet in cycle.

Complexity

Time: O(n) · Space: O(1)

Algorithm

- slow,fast
- Advance
- Check

Java Template

```
Floyd cycle
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Fast/Slow Pointer (List) Easy** (Easy): Canonical easy problem — Floyd algorithm.

Medium Examples

- **Fast/Slow Pointer (List) Medium** (Medium): Interview variant — Floyd algorithm.

Hard / Interview Examples

- **Fast/Slow Pointer (List) Hard** (Hard): Google-style twist — Floyd algorithm.

30-Second Recognition: Triggers: cycle, middle

> Trap: Using Fast/Slow Pointer (List) when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#18 LINKED LIST Mastered (+10 XP)

Merge Linked Lists

Why does Merge Linked Lists work? Merge two sorted lists with dummy head.

Mental Model: Zipper two sorted streams.

Recognition Signals: "merge two lists", "sorted lists"

When to use: Merge k=2 sorted linked lists.

When NOT to use: Unsorted merge.

Brute Force

Compare all pairs.

Optimal Approach

Compare and attach smaller.

Key Observation

Dummy node simplifies head.

Complexity

Time: `O(n+m)` · Space: `O(1)`

Algorithm

- dummy
- while both
- attach rest

Java Template

```
ListNode dummy=new ListNode(0),t=dummy;
while(l1!=null&&l2!=null){...}
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Merge Linked Lists Easy** (Easy): Canonical easy problem — Compare and attach smaller.

Medium Examples

- **Merge Linked Lists Medium** (Medium): Interview variant — Compare and attach smaller.

Hard / Interview Examples

- **Merge Linked Lists Hard** (Hard): Google-style twist — Compare and attach smaller.

30-Second Recognition: Triggers: merge two lists, sorted lists

> Trap: Using Merge Linked Lists when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#19 LINKED LIST Mastered (+10 XP)

Cycle Detection

Why does Cycle Detection work? Floyd + optional phase 2 for entry.

Mental Model: Phase1 meet, phase2 find entry.

Recognition Signals: "cycle start", "duplicate"

When to use: Detect cycle and find start.

When NOT to use: DAG.

Brute Force

Visited set.

Optimal Approach

Reset slow to head.

Key Observation

Distance math after meet.

Complexity

Time: O(n) · Space: O(1)

Algorithm

- Phase1
- Phase2

Java Template

```
Floyd entry
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Cycle Detection Easy** (Easy): Canonical easy problem — Reset slow to head.

Medium Examples

- **Cycle Detection Medium** (Medium): Interview variant — Reset slow to head.

Hard / Interview Examples

- **Cycle Detection Hard** (Hard): Google-style twist — Reset slow to head.

30-Second Recognition: Triggers: cycle start, duplicate

> Trap: Using Cycle Detection when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#20 LINKED LIST Mastered (+10 XP)

Linked List Rewiring

Why does Linked List Rewiring work? Reorder, rotate, swap nodes by changing pointers.

Mental Model: In-place pointer surgery.

Recognition Signals: "reorder list", "rotate list"

When to use: Complex reorder without extra array.

When NOT to use: Need stable array access.

Brute Force

Copy to array.

Optimal Approach

Find mid, reverse, merge.

Key Observation

Find middle, reverse second half.

Complexity

Time: O(n) · Space: O(1)

Algorithm

- mid
- reverse
- merge

Java Template

```
Reorder pattern
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Linked List Rewiring Easy** (Easy): Canonical easy problem — Find mid, reverse, merge.

Medium Examples

- **Linked List Rewiring Medium** (Medium): Interview variant — Find mid, reverse, merge.

Hard / Interview Examples

- **Linked List Rewiring Hard** (Hard): Google-style twist — Find mid, reverse, merge.

30-Second Recognition: Triggers: reorder list, rotate list

> Trap: Using Linked List Rewiring when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

STACK / QUEUE

#21 STACK / QUEUE Mastered (+10 XP)

Monotonic Stack

- Why does Monotonic Stack work? Stack maintaining increasing/decreasing order for next greater/smaller.

Mental Model: Stack of candidates — pop when current is better answer.

Recognition Signals: "next greater", "next smaller", "daily temperatures", "histogram"

When to use: Next greater/smaller element or rectangle in histogram.

When NOT to use: Need arbitrary access not nearest.

Brute Force

Brute next for each $O(n^2)$.

Optimal Approach

Push index; pop while violates monotonicity.

Key Observation

When popping, current is answer for popped index.

Complexity

Time: $O(n)$ · Space: $O(n)$

Algorithm

- Stack
- For i: while !empty && cond: pop ans[pop]=i
- Push i

Java Template

```
Deque<Integer> st=new ArrayDeque<>();
for(int i=0;i<n;i++){while(!st.isEmpty()&&nums[st.peek()]<nums[i]){...}st.push(i);}
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Monotonic Stack Easy** (Easy): Canonical easy problem — Push index; pop while violates monotonicity.

Medium Examples

- **Monotonic Stack Medium** (Medium): Interview variant — Push index; pop while violates monotonicity.

Hard / Interview Examples

- **Monotonic Stack Hard** (Hard): Google-style twist — Push index; pop while violates monotonicity.

30-Second Recognition: Triggers: next greater, next smaller, daily temperatures

> Trap: Using Monotonic Stack when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#22 STACK / QUEUE Mastered (+10 XP)

Monotonic Queue

Why does Monotonic Queue work? Deque storing useful candidates for sliding window min/max.

Mental Model: Queue drops useless elements.

Recognition Signals: "sliding window maximum", "window min"

When to use: Window min/max in O(n).

When NOT to use: Static array without sliding.

Brute Force

Heap per window O(n log k).

Optimal Approach

Pop back while worse; pop front if out of window.

Key Observation

Front always current window optimum.

Complexity

Time: $O(n)$ · Space: $O(k)$

Algorithm

- Deque
- Maintain decreasing
- Record max

Java Template

```
Deque<Integer> dq=new ArrayDeque<>();
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Monotonic Queue Easy** (Easy): Canonical easy problem — Pop back while worse; pop front if out of window.

Medium Examples

- **Monotonic Queue Medium** (Medium): Interview variant — Pop back while worse; pop front if out of window.

Hard / Interview Examples

- **Monotonic Queue Hard** (Hard): Google-style twist — Pop back while worse; pop front if out of window.

30-Second Recognition: Triggers: sliding window maximum, window min

> Trap: Using Monotonic Queue when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#23 STACK / QUEUE Mastered (+10 XP)

Stack Simulation

Why does Stack Simulation work? Simulate process with stack (cars, asteroids).

Mental Model: Stack models LIFO processing order.

Recognition Signals: "simulate", "stack", "remove adjacent"

When to use: Sequential removal/processing.

When NOT to use: Queue order needed.

Brute Force

Recursion.

Optimal Approach

Push/pop based on rule.

Key Observation

Top of stack interacts with current.

Complexity

Time: $O(n)$ · Space: $O(n)$

Algorithm

- Stack
- Process each
- Pop while condition

Java Template

```
Deque<Integer> st=new ArrayDeque<>();
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Stack Simulation Easy** (Easy): Canonical easy problem — Push/pop based on rule.

Medium Examples

- **Stack Simulation Medium** (Medium): Interview variant — Push/pop based on rule.

Hard / Interview Examples

- **Stack Simulation Hard** (Hard): Google-style twist — Push/pop based on rule.

30-Second Recognition: Triggers: simulate, stack, remove adjacent

> Trap: Using Stack Simulation when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#24 STACK / QUEUE Mastered (+10 XP)

Expression / Parentheses

Why does Expression / Parentheses work? Validate/match parentheses and evaluate expressions.

Mental Model: Stack tracks open brackets/operators.

Recognition Signals: "valid parentheses", "calculator", "decode string"

When to use: Bracket matching, nested structure.

When NOT to use: No nesting.

Brute Force

Counter only for single type.

Optimal Approach

Push open; pop on close.

Key Observation

Mismatch on pop.

Complexity

Time: O(n) · Space: O(n)

Algorithm

- Stack
- Map pairs
- Evaluate

Java Template

```
Stack<Character> st=new Stack<>();
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Expression / Parentheses Easy** (Easy): Canonical easy problem — Push open; pop on close.

Medium Examples

- **Expression / Parentheses Medium** (Medium): Interview variant — Push open; pop on close.

Hard / Interview Examples

- **Expression / Parentheses Hard** (Hard): Google-style twist — Push open; pop on close.

30-Second Recognition: Triggers: valid parentheses, calculator, decode string

> Trap: Using Expression / Parentheses when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#25 STACK / QUEUE Mastered (+10 XP)

BFS Queue Pattern

Why does BFS Queue Pattern work? FIFO layer-by-layer exploration.

Mental Model: Queue processes frontier.

Recognition Signals: "level order", "shortest steps", "BFS"

When to use: Shortest path unweighted, level traversal.

When NOT to use: DFS sufficient and simpler.

Brute Force

DFS with depth tracking.

Optimal Approach

Enqueue neighbors.

Key Observation

First time reached = shortest in unweighted.

Complexity

Time: O(V+E) · Space: O(V)

Algorithm

- Queue
- Mark visited
- Poll and expand

Java Template

```
Queue<int[]> q = new ArrayDeque<>();
q.offer(new int[]{sr, sc});
boolean[][] vis = new boolean[rows][cols];
vis[sr][sc] = true;
while (!q.isEmpty()) {
  int[] cur = q.poll();
  for (int[] d : dirs) { /* neighbor logic */ }
}
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **BFS Queue Pattern Easy** (Easy): Canonical easy problem — Enqueue neighbors.

Medium Examples

- **BFS Queue Pattern Medium** (Medium): Interview variant — Enqueue neighbors.

Hard / Interview Examples

- **BFS Queue Pattern Hard** (Hard): Google-style twist — Enqueue neighbors.

30-Second Recognition: Triggers: level order, shortest steps, BFS

> Trap: Using BFS Queue Pattern when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

HEAP

#26 HEAP Mastered (+10 XP)

Top K

Why does Top K work? Min-heap of size k for k largest; max-heap for k smallest.

Mental Model: Heap keeps only k best candidates.

Recognition Signals: "k largest", "kth element", "top k frequent"

When to use: Find/maintain top K elements.

When NOT to use: K=1 or full sort easier.

Brute Force

Sort O(n log n).

Optimal Approach

Min-heap size k for k largest.

Key Observation

Heap size k discards losers.

Complexity

Time: `O(n log k)` · Space: `O(k)`

Algorithm

- Heap
- Maintain size k
- Poll if exceed

Java Template

```
PriorityQueue<Integer> minHeap=new PriorityQueue<>();
for(int x:nums){minHeap.offer(x);if(minHeap.size()>k)minHeap.poll();}
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Top K Easy** (Easy): Canonical easy problem — Min-heap size k for k largest.

Medium Examples

- **Top K Medium** (Medium): Interview variant — Min-heap size k for k largest.

Hard / Interview Examples

- **Top K Hard** (Hard): Google-style twist — Min-heap size k for k largest.

30-Second Recognition: Triggers: k largest, kth element, top k frequent

> Trap: Using Top K when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#27 HEAP ☐ Mastered (+10 XP)

K-way Merge

Why does K-way Merge work? Merge k sorted lists/arrays with heap of heads.

Mental Model: Heap picks smallest among k current heads.

Recognition Signals: "merge k lists", "k sorted"

When to use: Merge k sorted sequences.

When NOT to use: k=2 use two pointer.

Brute Force

Concatenate and sort.

Optimal Approach

Heap of (value, list_id, index).

Key Observation

Each step pick min of k heads.

Complexity

Time: `O(N log k)` · Space: `O(k)`

Algorithm

- Init heap with first of each
- Poll min, push next

Java Template

```
PriorityQueue<int[]> pq=new PriorityQueue<>((a,b)->a[0]-b[0]);
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **K-way Merge Easy** (Easy): Canonical easy problem — Heap of (value, list_id, index).

Medium Examples

- **K-way Merge Medium** (Medium): Interview variant — Heap of (value, list_id, index).

Hard / Interview Examples

- **K-way Merge Hard** (Hard): Google-style twist — Heap of (value, list_id, index).

30-Second Recognition: Triggers: merge k lists, k sorted

> Trap: Using K-way Merge when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#28 HEAP Mastered (+10 XP)

Two Heaps

Why does Two Heaps work? Max-heap lower half + min-heap upper half for streaming median.

Mental Model: Balance sizes for median.

Recognition Signals: "find median", "stream median"

When to use: Dynamic median from stream.

When NOT to use: Offline median — sort.

Brute Force

Resort each insert.

Optimal Approach

Push then rebalance.

Key Observation

Keep heaps balanced ±1.

Complexity

Time: `O(log n) per insert` · Space: `O(n)`

Algorithm

- maxHeap low
- minHeap high
- balance

Java Template

```
Two PriorityQueues
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Two Heaps Easy** (Easy): Canonical easy problem — Push then rebalance.

Medium Examples

- **Two Heaps Medium** (Medium): Interview variant — Push then rebalance.

Hard / Interview Examples

- **Two Heaps Hard** (Hard): Google-style twist — Push then rebalance.

30-Second Recognition: Triggers: find median, stream median

> Trap: Using Two Heaps when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#29 HEAP Mastered (+10 XP)

Scheduling with Heap

Why does Scheduling with Heap work? Greedy pick earliest finish/available resource with heap.

Mental Model: Heap of available times or tasks.

Recognition Signals: "meeting rooms", "cpu tasks", "schedule"

When to use: Task scheduling with priorities.

When NOT to use: No priority constraints.

Brute Force

Brute assign.

Optimal Approach

Heap by deadline/time.

Key Observation

Always process most urgent.

Complexity

Time: $O(n \log n)$ · Space: $O(n)$

Algorithm

- Heap
- Greedy pop
- Assign

Java Template

```
PriorityQueue by deadline
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Scheduling with Heap Easy** (Easy): Canonical easy problem — Heap by deadline/time.

Medium Examples

- **Scheduling with Heap Medium** (Medium): Interview variant — Heap by deadline/time.

Hard / Interview Examples

- **Scheduling with Heap Hard** (Hard): Google-style twist — Heap by deadline/time.

30-Second Recognition: Triggers: meeting rooms, cpu tasks, schedule

> Trap: Using Scheduling with Heap when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#30 HEAP Mastered (+10 XP)

Heap + Greedy

Why does Heap + Greedy work? Combine greedy choice with heap for optimal local picks.

Mental Model: Greedy with efficient best pick.

Recognition Signals: "task scheduler", "reorganize string"

When to use: Frequency/scheduling greedy.

When NOT to use: DP optimal needed.

Brute Force

Try all orders.

Optimal Approach

Poll two highest freq.

Key Observation

Heap gives max freq char.

Complexity

Time: $O(n \log k)$ · Space: $O(k)$

Algorithm

- Freq map
- Max heap
- Greedy build

Java Template

```
PriorityQueue<int[]> pq
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Heap + Greedy Easy** (Easy): Canonical easy problem — Poll two highest freq.

Medium Examples

- **Heap + Greedy Medium** (Medium): Interview variant — Poll two highest freq.

Hard / Interview Examples

- **Heap + Greedy Hard** (Hard): Google-style twist — Poll two highest freq.

30-Second Recognition: Triggers: task scheduler, reorganize string

> Trap: Using Heap + Greedy when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

TREE / BST

#31 TREE / BST ☐ Mastered (+10 XP)

DFS Tree Traversal

Why does DFS Tree Traversal work? Preorder/inorder/postorder recursive or iterative.

Mental Model: Go deep before backtracking.

Recognition Signals: "tree traversal", "path sum", "subtree"

When to use: Explore tree paths/subtrees.

When NOT to use: Shortest path level — use BFS.

Brute Force

Level-by-level recursion overhead.

Optimal Approach

Recursive or stack iterative.

Key Observation

Base case null.

Complexity

Time: $O(n)$ · Space: $O(h)$

Algorithm

- Base null
- Process
- Recurse children

Java Template

```
void dfs(TreeNode n){if(n==null)return;dfs(n.left);dfs(n.right);}
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **DFS Tree Traversal Easy** (Easy): Canonical easy problem — Recursive or stack iterative.

Medium Examples

- **DFS Tree Traversal Medium** (Medium): Interview variant — Recursive or stack iterative.

Hard / Interview Examples

- **DFS Tree Traversal Hard** (Hard): Google-style twist — Recursive or stack iterative.

30-Second Recognition: Triggers: tree traversal, path sum, subtree

> Trap: Using DFS Tree Traversal when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#32 TREE / BST ☐ Mastered (+10 XP)

BFS / Level Order

Why does BFS / Level Order work? Queue processes tree level by level.

Mental Model: Layer cake traversal.

Recognition Signals: "level order", "zigzag", "right side view"

When to use: Level-by-level or shortest in tree.

When NOT to use: Deep path only — DFS fine.

Brute Force

DFS with depth.

Optimal Approach

For each level process all in queue.

Key Observation

Queue size = level width.

Complexity

Time: $O(n)$ · Space: $O(w)$

Algorithm

- Queue root
- While !empty: level size loop

Java Template

```
Queue<TreeNode> q=new ArrayDeque<>();q.offer(root);
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **BFS / Level Order Easy** (Easy): Canonical easy problem — For each level process all in queue.

Medium Examples

- **BFS / Level Order Medium** (Medium): Interview variant — For each level process all in queue.

Hard / Interview Examples

- **BFS / Level Order Hard** (Hard): Google-style twist — For each level process all in queue.

30-Second Recognition: Triggers: level order, zigzag, right side view

> Trap: Using BFS / Level Order when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#33 TREE / BST Mastered (+10 XP)

Recursive Tree DP

Why does Recursive Tree DP work? Post-order DP on tree nodes.

Mental Model: Answer at node from children answers.

Recognition Signals: "house robber III", "tree DP"

When to use: Optimal on tree with child dependency.

When NOT to use: Linear DP on array.

Brute Force

Recalculate subtrees.

Optimal Approach

Return (with, without) from children.

Key Observation

post-order combine.

Complexity

Time: $O(n)$ · Space: $O(h)$

Algorithm

- DFS postorder
- Combine child states

Java Template

```
int[] rob(TreeNode n){if(n==null)return new int[]{0,0};}
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Recursive Tree DP Easy** (Easy): Canonical easy problem — Return (with, without) from children.

Medium Examples

- **Recursive Tree DP Medium** (Medium): Interview variant — Return (with, without) from children.

Hard / Interview Examples

- **Recursive Tree DP Hard** (Hard): Google-style twist — Return (with, without) from children.

30-Second Recognition: Triggers: house robber III, tree DP

> Trap: Using Recursive Tree DP when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#34 TREE / BST Mastered (+10 XP)

Path / Diameter

Why does Path / Diameter work? Track path through node; global max diameter.

Mental Model: Height at each node contributes paths.

Recognition Signals: "diameter", "max path sum"

When to use: Longest path or max path sum.

When NOT to use: All paths enumeration.

Brute Force

Try all pairs leaves.

Optimal Approach

Postorder height update global max.

Key Observation

Diameter = left_h + right_h.

Complexity

Time: $O(n)$ · Space: $O(h)$

Algorithm

- DFS return height
- Update ans with left+right

Java Template

```
int dfs(TreeNode n){if(n==null)return 0;int l=dfs(n.left);...}
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Path / Diameter Easy** (Easy): Canonical easy problem — Postorder height update global max.

Medium Examples

- **Path / Diameter Medium** (Medium): Interview variant — Postorder height update global max.

Hard / Interview Examples

- **Path / Diameter Hard** (Hard): Google-style twist — Postorder height update global max.

30-Second Recognition: Triggers: diameter, max path sum

> Trap: Using Path / Diameter when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#35 TREE / BST ☐ Mastered (+10 XP)

Lowest Common Ancestor

Why does Lowest Common Ancestor work? BST: split point; Binary tree: postorder match.

Mental Model: First node where p,q diverge.

Recognition Signals: "LCA", "lowest common ancestor"

When to use: Find LCA of two nodes.

When NOT to use: Single node.

Brute Force

Store paths to root.

Optimal Approach

Recursive return if found.

Key Observation

If p,q on different sides current is LCA.

Complexity

Time: $O(n)$ · Space: $O(h)$

Algorithm

- If null or p or q return
- left,right
- both non-null -> root

Java Template

```
TreeNode lca(TreeNode r,TreeNode p,TreeNode q){...}
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Lowest Common Ancestor Easy** (Easy): Canonical easy problem — Recursive return if found.

Medium Examples

- **Lowest Common Ancestor Medium** (Medium): Interview variant — Recursive return if found.

Hard / Interview Examples

- **Lowest Common Ancestor Hard** (Hard): Google-style twist — Recursive return if found.

30-Second Recognition: Triggers: LCA, lowest common ancestor

> Trap: Using Lowest Common Ancestor when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#36 TREE / BST ☐ Mastered (+10 XP)

BST Invariants

Why does BST Invariants work? Use sorted property: inorder, bounds checking.

Mental Model: Left < root < right everywhere.

Recognition Signals: "validate BST", "kth smallest"

When to use: BST search and validation.

When NOT to use: General binary tree.

Brute Force

Sort values.

Optimal Approach

Bounds min,max per node.

Key Observation

Inorder is sorted.

Complexity

Time: O(n) · Space: O(h)

Algorithm

- Check bounds
- Recurse

Java Template

```
boolean valid(TreeNode n,long min,long max)
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **BST Invariants Easy** (Easy): Canonical easy problem — Bounds min,max per node.

Medium Examples

- **BST Invariants Medium** (Medium): Interview variant — Bounds min,max per node.

Hard / Interview Examples

- **BST Invariants Hard** (Hard): Google-style twist — Bounds min,max per node.

30-Second Recognition: Triggers: validate BST, kth smallest

> Trap: Using BST Invariants when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#37 TREE / BST Mastered (+10 XP)

Tree Serialization

Why does Tree Serialization work? Preorder with null markers or BFS.

Mental Model: Encode structure for storage.

Recognition Signals: "serialize", "deserialize"

When to use: Persist tree structure.

When NOT to use: Values only no structure.

Brute Force

JSON with indices.

Optimal Approach

Preorder recurse.

Key Observation

Null markers preserve shape.

Complexity

Time: O(n) · Space: O(n)

Algorithm

- Build string
- Parse with index

Java Template

```
String serialize(TreeNode root)
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Tree Serialization Easy** (Easy): Canonical easy problem — Preorder recurse.

Medium Examples

- **Tree Serialization Medium** (Medium): Interview variant — Preorder recurse.

Hard / Interview Examples

- **Tree Serialization Hard** (Hard): Google-style twist — Preorder recurse.

30-Second Recognition: Triggers: serialize, deserialize

> Trap: Using Tree Serialization when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

GRAPH

#38 GRAPH Mastered (+10 XP)

BFS/DFS Grid

Why does BFS/DFS Grid work? 2D matrix as implicit graph.

Mental Model: 4/8 directions on grid.

Recognition Signals: "island", "grid", "matrix BFS"

When to use: Connected components on grid.

When NOT to use: 1D array.

Brute Force

Visit all cells repeatedly.

Optimal Approach

Enqueue 4 neighbors.

Key Observation

Mark visited in-place or set.

Complexity

Time: $O(mn)$ · Space: $O(mn)$

Algorithm

- Dirs array
- Mark visited
- BFS/DFS

Java Template

```
Queue<int[]> q = new ArrayDeque<>();
q.offer(new int[]{sr, sc});
boolean[][] vis = new boolean[rows][cols];
vis[sr][sc] = true;
while (!q.isEmpty()) {
  int[] cur = q.poll();
  for (int[] d : dirs) { /* neighbor logic */ }
}
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **BFS/DFS Grid Easy** (Easy): Canonical easy problem — Enqueue 4 neighbors.

Medium Examples

- **BFS/DFS Grid Medium** (Medium): Interview variant — Enqueue 4 neighbors.

Hard / Interview Examples

- **BFS/DFS Grid Hard** (Hard): Google-style twist — Enqueue 4 neighbors.

30-Second Recognition: Triggers: island, grid, matrix BFS

> Trap: Using BFS/DFS Grid when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#39 GRAPH ☐ Mastered (+10 XP)

Graph Traversal

Why does Graph Traversal work? Adjacency list BFS/DFS for connectivity.

Mental Model: Visit all reachable nodes.

Recognition Signals: "connected components", "graph"

When to use: Explore graph.

When NOT to use: Tree only — simpler.

Brute Force

Repeated scans.

Optimal Approach

For each unvisited start DFS.

Key Observation

Visited array.

Complexity

Time: O(V+E) · Space: O(V)

Algorithm

- Build adj
- Loop nodes
- DFS/BFS

Java Template

```
void dfs(int u) {
  vis[u] = true;
  for (int v : adj.get(u)) if (!vis[v]) dfs(v);
}
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Graph Traversal Easy** (Easy): Canonical easy problem — For each unvisited start DFS.

Medium Examples

- **Graph Traversal Medium** (Medium): Interview variant — For each unvisited start DFS.

Hard / Interview Examples

- **Graph Traversal Hard** (Hard): Google-style twist — For each unvisited start DFS.

30-Second Recognition: Triggers: connected components, graph

> Trap: Using Graph Traversal when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#40 GRAPH Mastered (+10 XP)

Cycle Detection (Graph)

Why does Cycle Detection (Graph) work? DFS with 3-color or Union-Find.

Mental Model: Back edge = cycle in directed.

Recognition Signals: "course schedule", "cycle"

When to use: Detect cycle directed/undirected.

When NOT to use: DAG topological sort.

Brute Force

Try all paths.

Optimal Approach

DFS coloring.

Key Observation

Gray node revisited = cycle.

Complexity

Time: $O(V+E)$ · Space: $O(V)$

Algorithm

- WHITE/GRAY/BLACK
- DFS

Java Template

```
boolean hasCycle(int u)
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Cycle Detection (Graph) Easy** (Easy): Canonical easy problem — DFS coloring.

Medium Examples

- **Cycle Detection (Graph) Medium** (Medium): Interview variant — DFS coloring.

Hard / Interview Examples

- **Cycle Detection (Graph) Hard** (Hard): Google-style twist — DFS coloring.

30-Second Recognition: Triggers: course schedule, cycle

> Trap: Using Cycle Detection (Graph) when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#41 GRAPH ☐ Mastered (+10 XP)

Topological Sort

Why does Topological Sort work? Ordering with all edges forward — Kahn or DFS postorder.

Mental Model: Dependencies first.

Recognition Signals: "course schedule", "dependency"

When to use: DAG ordering.

When NOT to use: Cycle exists — impossible.

Brute Force

Brute all permutations.

Optimal Approach

Kahn BFS.

Key Observation

In-degree 0 queue.

Complexity

Time: O(V+E) · Space: O(V)

Algorithm

- In-degree
- Queue zeros
- Reduce

Java Template

```
Queue<Integer> q; // Kahn
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Topological Sort Easy** (Easy): Canonical easy problem — Kahn BFS.

Medium Examples

- **Topological Sort Medium** (Medium): Interview variant — Kahn BFS.

Hard / Interview Examples

- **Topological Sort Hard** (Hard): Google-style twist — Kahn BFS.

30-Second Recognition: Triggers: course schedule, dependency

> Trap: Using Topological Sort when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#42 GRAPH Mastered (+10 XP)

Union Find / DSU

Why does Union Find / DSU work? Disjoint set with path compression + rank.

Mental Model: Groups merge; ask same set.

Recognition Signals: "connected components", "redundant connection"

When to use: Dynamic connectivity.

When NOT to use: Static graph one-time DFS enough.

Brute Force

BFS each query.

Optimal Approach

find with compression; union by rank.

Key Observation

Path compression flattens tree.

Complexity

Time: `α(n) amortized` · Space: `O(n)`

Algorithm

- parent[], rank[]
- find
- union

Java Template

```
class UF{int[]p,r;int find(int x){...}void union(int a,int b){...}}
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Union Find / DSU Easy** (Easy): Canonical easy problem — find with compression; union by rank.

Medium Examples

- **Union Find / DSU Medium** (Medium): Interview variant — find with compression; union by rank.

Hard / Interview Examples

- **Union Find / DSU Hard** (Hard): Google-style twist — find with compression; union by rank.

30-Second Recognition: Triggers: connected components, redundant connection

> Trap: Using Union Find / DSU when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#43 GRAPH Mastered (+10 XP)

Shortest Path — BFS

Why does Shortest Path — BFS work? Unweighted shortest path — BFS levels.

Mental Model: Each edge weight 1.

Recognition Signals: "shortest path", "minimum steps", "unweighted"

When to use: Unweighted shortest path.

When NOT to use: Weighted edges.

Brute Force

DFS may not be shortest.

Optimal Approach

BFS with distance array.

Key Observation

First visit = shortest.

Complexity

Time: O(V+E) · Space: O(V)

Algorithm

- Queue
- dist[]
- Expand

Java Template

```
Queue<int[]> q = new ArrayDeque<>();
q.offer(new int[]{sr, sc});
boolean[][] vis = new boolean[rows][cols];
vis[sr][sc] = true;
while (!q.isEmpty()) {
  int[] cur = q.poll();
  for (int[] d : dirs) { /* neighbor logic */ }
}
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Shortest Path — BFS Easy** (Easy): Canonical easy problem — BFS with distance array.

Medium Examples

- **Shortest Path — BFS Medium** (Medium): Interview variant — BFS with distance array.

Hard / Interview Examples

- **Shortest Path — BFS Hard** (Hard): Google-style twist — BFS with distance array.

30-Second Recognition: Triggers: shortest path, minimum steps, unweighted

> Trap: Using Shortest Path — BFS when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#44 GRAPH Mastered (+10 XP)

Dijkstra

Why does Dijkstra work? Non-negative weighted shortest path with min-heap.

Mental Model: Greedy expand closest unvisited.

Recognition Signals: "weighted graph", "network delay"

When to use: Non-negative edge weights.

When NOT to use: Negative weights.

Brute Force

Bellman-Ford always.

Optimal Approach

Heap of (dist, node).

Key Observation

Settled node distance is final.

Complexity

Time: `O((V+E) log V)` · Space: `O(V)`

Algorithm

- dist[]
- PQ
- Relax edges

Java Template

```
PriorityQueue<long[]> pq=new PriorityQueue<>((a,b)->Long.compare(a[0],b[0]));
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Dijkstra Easy** (Easy): Canonical easy problem — Heap of (dist, node).

Medium Examples

- **Dijkstra Medium** (Medium): Interview variant — Heap of (dist, node).

Hard / Interview Examples

- **Dijkstra Hard** (Hard): Google-style twist — Heap of (dist, node).

30-Second Recognition: Triggers: weighted graph, network delay

> Trap: Using Dijkstra when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#45 GRAPH Mastered (+10 XP)

MST — Kruskal / Prim

Why does MST — Kruskal / Prim work? Minimum spanning tree — sort edges or grow from node.

Mental Model: Connect all min total weight.

Recognition Signals: "min cost", "connect all"

When to use: MST in undirected weighted graph.

When NOT to use: Shortest path not MST.

Brute Force

Try all spanning trees.

Optimal Approach

Prim: heap grow.

Key Observation

Kruskal: UF + sort edges.

Complexity

Time: $O(E \log E)$ · Space: $O(V)$

Algorithm

- Sort edges UF
- Or Prim PQ

Java Template

```
Kruskal with UF
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **MST — Kruskal / Prim Easy** (Easy): Canonical easy problem — Prim: heap grow.

Medium Examples

- **MST — Kruskal / Prim Medium** (Medium): Interview variant — Prim: heap grow.

Hard / Interview Examples

- **MST — Kruskal / Prim Hard** (Hard): Google-style twist — Prim: heap grow.

30-Second Recognition: Triggers: min cost, connect all

> Trap: Using MST — Kruskal / Prim when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

BACKTRACKING

#46 BACKTRACKING ☐ Mastered (+10 XP)

Subsets

Why does Subsets work? Include/exclude each element.

Mental Model: Binary choice tree.

Recognition Signals: "subsets", "power set"

When to use: All subsets.

When NOT to use: Need count only — 2^n formula.

Brute Force

Iterative copy.

Optimal Approach

backtrack(idx).

Key Observation

Each level decide include.

Complexity

Time: O(n*2^n) · Space: O(n)

Algorithm

- path
- for i from idx
- choose/not

Java Template

```
void bt(int i){if(i==n){res.add(new ArrayList<>(path));return;}
path.add(nums[i]);bt(i+1);path.remove(path.size()-1);bt(i+1);}
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Subsets Easy** (Easy): Canonical easy problem — backtrack(idx).

Medium Examples

- **Subsets Medium** (Medium): Interview variant — backtrack(idx).

Hard / Interview Examples

- **Subsets Hard** (Hard): Google-style twist — backtrack(idx).

30-Second Recognition: Triggers: subsets, power set

> Trap: Using Subsets when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#47 BACKTRACKING Mastered (+10 XP)

Permutations

Why does Permutations work? Swap or used[] to build all orderings.

Mental Model: Fill position by position.

Recognition Signals: "permutations", "arrangements"

When to use: All orderings.

When NOT to use: Need subset not order.

Brute Force

Next permutation only.

Optimal Approach

backtrack with used.

Key Observation

used array prevents reuse.

Complexity

Time: O(n!) · Space: O(n)

Algorithm

- used[]
- path
- base n

Java Template

```
void bt(){if(path.size()==n){...}for...}
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Permutations Easy** (Easy): Canonical easy problem — backtrack with used.

Medium Examples

- **Permutations Medium** (Medium): Interview variant — backtrack with used.

Hard / Interview Examples

- **Permutations Hard** (Hard): Google-style twist — backtrack with used.

30-Second Recognition: Triggers: permutations, arrangements

> Trap: Using Permutations when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#48 BACKTRACKING Mastered (+10 XP)

Combination Sum

Why does Combination Sum work? Reuse allowed — same idx can repeat.

Mental Model: Unbounded knapsack style search.

Recognition Signals: "combination sum", "candidates"

When to use: Combinations summing to target.

When NOT to use: Distinct elements only.

Brute Force

Nested loops depth.

Optimal Approach

bt(idx, remain).

Key Observation

Start idx prevents permutations.

Complexity

Time: `O(2^target)` · Space: `O(target)`

Algorithm

- if remain==0 add
- for i from idx

Java Template

```
void bt(int i,int rem){...}
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Combination Sum Easy** (Easy): Canonical easy problem — bt(idx, remain).

Medium Examples

- **Combination Sum Medium** (Medium): Interview variant — bt(idx, remain).

Hard / Interview Examples

- **Combination Sum Hard** (Hard): Google-style twist — bt(idx, remain).

30-Second Recognition: Triggers: combination sum, candidates

> Trap: Using Combination Sum when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#49 BACKTRACKING Mastered (+10 XP)

Constraint Search / Pruning

Why does Constraint Search / Pruning work? Cut branches that violate constraints early.

Mental Model: Don't explore doomed paths.

Recognition Signals: "N-queens", "sudoku"

When to use: Heavy constraint satisfaction.

When NOT to use: Loose constraints brute ok.

Brute Force

Full search.

Optimal Approach

Prune on failure.

Key Observation

Check valid before recurse.

Complexity

Time: $O(n!)$ · Space: $O(n)$

Algorithm

- isValid
- place
- backtrack

Java Template

```
N-Queens bt
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Constraint Search / Pruning Easy** (Easy): Canonical easy problem — Prune on failure.

Medium Examples

- **Constraint Search / Pruning Medium** (Medium): Interview variant — Prune on failure.

Hard / Interview Examples

- **Constraint Search / Pruning Hard** (Hard): Google-style twist — Prune on failure.

30-Second Recognition: Triggers: N-queens, sudoku

> Trap: Using Constraint Search / Pruning when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

GREEDY

#50 GREEDY Mastered (+10 XP)

Greedy with Proof

Why does Greedy with Proof work? Local optimal via exchange argument.

Mental Model: Prove swapping never hurts.

Recognition Signals: "jump game", "gas station"

When to use: Greedy optimal with proof.

When NOT to use: Overlapping subproblems — DP.

Brute Force

Try all.

Optimal Approach

Track farthest reachable.

Key Observation

Exchange argument.

Complexity

Time: O(n) · Space: O(1)

Algorithm

- Greedy measure
- Update
- Prove

Java Template

```
int far=0;for(...)
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Greedy with Proof Easy** (Easy): Canonical easy problem — Track farthest reachable.

Medium Examples

- **Greedy with Proof Medium** (Medium): Interview variant — Track farthest reachable.

Hard / Interview Examples

- **Greedy with Proof Hard** (Hard): Google-style twist — Track farthest reachable.

30-Second Recognition: Triggers: jump game, gas station

> Trap: Using Greedy with Proof when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#51 GREEDY Mastered (+10 XP)

Scheduling / Interval Greedy

Why does Scheduling / Interval Greedy work? Sort by end time; pick non-overlapping max.

Mental Model: Earliest finish leaves room.

Recognition Signals: "activity selection", "meeting rooms"

When to use: Interval scheduling greedy.

When NOT to use: Weighted intervals — DP.

Brute Force

All subsets.

Optimal Approach

Greedy pick.

Key Observation

Sort by end.

Complexity

Time: `O(n log n)` · Space: `O(1)`

Algorithm

- Sort end
- Take if start>=lastEnd

Java Template

```
Activity selection
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Scheduling / Interval Greedy Easy** (Easy): Canonical easy problem — Greedy pick.

Medium Examples

- **Scheduling / Interval Greedy Medium** (Medium): Interview variant — Greedy pick.

Hard / Interview Examples

- **Scheduling / Interval Greedy Hard** (Hard): Google-style twist — Greedy pick.

30-Second Recognition: Triggers: activity selection, meeting rooms

> Trap: Using Scheduling / Interval Greedy when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

DYNAMIC PROGRAMMING

#52 DYNAMIC PROGRAMMING Mastered (+10 XP)

1D DP

Why does 1D DP work? dp[i] = best answer using first i elements.

Mental Model: Linear table of subproblems.

Recognition Signals: "climbing stairs", "house robber", "decode ways"

When to use: Optimal substructure linear.

When NOT to use: No overlapping states.

Brute Force

Recursion exponential.

Optimal Approach

Tabulation or memo.

Key Observation

dp[i] from dp[i-1], dp[i-2].

Complexity

Time: `O(n)` · Space: `O(n) or O(1)`

Algorithm

- Define dp
- Base cases
- Transition

Java Template

```
int[] dp=new int[n+1];dp[0]=1;
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **1D DP Easy** (Easy): Canonical easy problem — Tabulation or memo.

Medium Examples

- **1D DP Medium** (Medium): Interview variant — Tabulation or memo.

Hard / Interview Examples

- **1D DP Hard** (Hard): Google-style twist — Tabulation or memo.

30-Second Recognition: Triggers: climbing stairs, house robber, decode ways

> Trap: Using 1D DP when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#53 DYNAMIC PROGRAMMING Mastered (+10 XP)

2D/Grid DP

Why does 2D/Grid DP work? dp[r][c] from top/left neighbors.

Mental Model: Fill grid bottom-right.

Recognition Signals: "unique paths", "min path sum"

When to use: Grid path optimization.

When NOT to use: 1D sufficient.

Brute Force

Recurse all paths.

Optimal Approach

Nested loops.

Key Observation

Only from up and left.

Complexity

Time: $O(mn)$ · Space: $O(mn)$

Algorithm

- Init first row/col
- dp[r][c]=min(up,left)+val

Java Template

```
int[][] dp=new int[m][n];
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **2D/Grid DP Easy** (Easy): Canonical easy problem — Nested loops.

Medium Examples

- **2D/Grid DP Medium** (Medium): Interview variant — Nested loops.

Hard / Interview Examples

- **2D/Grid DP Hard** (Hard): Google-style twist — Nested loops.

30-Second Recognition: Triggers: unique paths, min path sum

> Trap: Using 2D/Grid DP when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#54 DYNAMIC PROGRAMMING Mastered (+10 XP)

Knapsack

Why does Knapsack work? 0/1: take or skip; unbounded: reuse.

Mental Model: Capacity dimension DP.

Recognition Signals: "knapsack", "subset sum", "partition"

When to use: Capacity/budget optimization.

When NOT to use: Unlimited greedy works.

Brute Force

Try all subsets.

Optimal Approach

Iterate items and weights.

Key Observation

dp[w] max value with weight w.

Complexity

Time: $O(nW)$ · Space: $O(W)$

Algorithm

- dp[0..W]
- For each item
- Reverse w for 0/1

Java Template

```
int[] dp=new int[cap+1];
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Knapsack Easy** (Easy): Canonical easy problem — Iterate items and weights.

Medium Examples

- **Knapsack Medium** (Medium): Interview variant — Iterate items and weights.

Hard / Interview Examples

- **Knapsack Hard** (Hard): Google-style twist — Iterate items and weights.

30-Second Recognition: Triggers: knapsack, subset sum, partition

> Trap: Using Knapsack when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#55 DYNAMIC PROGRAMMING Mastered (+10 XP)

Subsequence DP

Why does Subsequence DP work? LCS/LIS/Palindrome subsequence.

Mental Model: Two sequence or LIS patience.

Recognition Signals: "LCS", "LIS", "palindrome subseq"

When to use: Subsequence not substring.

When NOT to use: Contiguous — sliding window.

Brute Force

Generate all subseq.

Optimal Approach

$O(n^2)$ or patience O(n log n) for LIS.

Key Observation

dp[i][j] match chars.

Complexity

Time: `O(nm)` · Space: `O(nm)`

Algorithm

- Define states
- Transitions

Java Template

```
LCS dp table
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Subsequence DP Easy** (Easy): Canonical easy problem — $O(n^2)$ or patience O(n log n) for LIS.

Medium Examples

- **Subsequence DP Medium** (Medium): Interview variant — $O(n^2)$ or patience O(n log n) for LIS.

Hard / Interview Examples

- **Subsequence DP Hard** (Hard): Google-style twist — $O(n^2)$ or patience O(n log n) for LIS.

30-Second Recognition: Triggers: LCS, LIS, palindrome subseq

> Trap: Using Subsequence DP when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#56 DYNAMIC PROGRAMMING Mastered (+10 XP)

State-machine DP

Why does State-machine DP work? dp[day][holding] stock problems.

Mental Model: Finite states per step.

Recognition Signals: "buy sell stock", "cooldown"

When to use: Stock with states hold/sold/cool.

When NOT to use: No state needed.

Brute Force

Brute all transactions.

Optimal Approach

dp[i][0/1].

Key Observation

State transitions limited.

Complexity

Time: $O(n)$ · Space: $O(1)$

Algorithm

- States
- Day loop
- Max profit

Java Template

```
int sold=0, hold=Integer.MIN_VALUE;
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **State-machine DP Easy** (Easy): Canonical easy problem — dp[i][0/1].

Medium Examples

- **State-machine DP Medium** (Medium): Interview variant — dp[i][0/1].

Hard / Interview Examples

- **State-machine DP Hard** (Hard): Google-style twist — dp[i][0/1].

30-Second Recognition: Triggers: buy sell stock, cooldown

> Trap: Using State-machine DP when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#57 DYNAMIC PROGRAMMING Mastered (+10 XP)

DP on Trees

Why does DP on Trees work? Postorder aggregate subtree answers.

Mental Model: Each node combines child DP.

Recognition Signals: "tree DP", "max path"

When to use: Tree optimization.

When NOT to use: Linear DP.

Brute Force

Recompute subtrees.

Optimal Approach

Return tuple from children.

Key Observation

Postorder.

Complexity

Time: $O(n)$ · Space: $O(h)$

Algorithm

- DFS
- Merge child results

Java Template

```
Tree DP postorder
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **DP on Trees Easy** (Easy): Canonical easy problem — Return tuple from children.

Medium Examples

- **DP on Trees Medium** (Medium): Interview variant — Return tuple from children.

Hard / Interview Examples

- **DP on Trees Hard** (Hard): Google-style twist — Return tuple from children.

30-Second Recognition: Triggers: tree DP, max path

> Trap: Using DP on Trees when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#58 DYNAMIC PROGRAMMING Mastered (+10 XP)

DP + Optimization

Why does DP + Optimization work? Monotonic deque/convex hull speed DP.

Mental Model: Reduce inner loop with structure.

Recognition Signals: "dp optimization", "sliding window max in DP"

When to use: DP transition bottleneck.

When NOT to use: Small constraints brute.

Brute Force

$O(n^2)$ DP.

Optimal Approach

Optimize transition.

Key Observation

Deque maintains optimal window.

Complexity

Time: $O(n)$ · Space: $O(n)$

Algorithm

- DP loop
- Deque helper

Java Template

```
Monotonic deque DP
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **DP + Optimization Easy** (Easy): Canonical easy problem — Optimize transition.

Medium Examples

- **DP + Optimization Medium** (Medium): Interview variant — Optimize transition.

Hard / Interview Examples

- **DP + Optimization Hard** (Hard): Google-style twist — Optimize transition.

30-Second Recognition: Triggers: dp optimization, sliding window max in DP

> Trap: Using DP + Optimization when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

ADVANCED

#59 ADVANCED Mastered (+10 XP)

Trie

Why does Trie work? Prefix tree for strings/words.

Mental Model: Character edges per level.

Recognition Signals: "prefix", "autocomplete", "word search"

When to use: Prefix queries, dictionary.

When NOT to use: Few strings hash enough.

Brute Force

Hash all prefixes.

Optimal Approach

Node children[26].

Key Observation

Shared prefixes compressed.

Complexity

Time: `O(L)` · Space: `O(total chars)`

Algorithm

- Insert
- Search
- startsWith

Java Template

```
class TrieNode{TrieNode[] ch=new TrieNode[26];}
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Trie Easy** (Easy): Canonical easy problem — Node children[26].

Medium Examples

- **Trie Medium** (Medium): Interview variant — Node children[26].

Hard / Interview Examples

- **Trie Hard** (Hard): Google-style twist — Node children[26].

30-Second Recognition: Triggers: prefix, autocomplete, word search

> Trap: Using Trie when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#60 ADVANCED Mastered (+10 XP)

Bit Manipulation

Why does Bit Manipulation work? XOR/AND tricks for parity, subsets, masks.

Mental Model: Bits encode state compactly.

Recognition Signals: "single number", "subset XOR", "power of two"

When to use: XOR cancel pairs; bit masks.

When NOT to use: Large numbers need BigInteger.

Brute Force

Hash count.

Optimal Approach

Bitmask DP or tricks.

Key Observation

a^a=0; n&(n-1) clears lowest bit.

Complexity

Time: O(n) · Space: O(1)

Algorithm

- XOR all
- n&=n-1
- 1<

Java Template

```
int xor=0;for(int x:nums)xor^=x;
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Bit Manipulation Easy** (Easy): Canonical easy problem — Bitmask DP or tricks.

Medium Examples

- **Bit Manipulation Medium** (Medium): Interview variant — Bitmask DP or tricks.

Hard / Interview Examples

- **Bit Manipulation Hard** (Hard): Google-style twist — Bitmask DP or tricks.

30-Second Recognition: Triggers: single number, subset XOR, power of two

> Trap: Using Bit Manipulation when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#61 ADVANCED Mastered (+10 XP)

Sweep Line

Why does Sweep Line work? Events sorted by coordinate process status.

Mental Model: Timeline of interval start/end.

Recognition Signals: "skyline", "meeting rooms II"

When to use: Count overlapping intervals over time.

When NOT to use: Few intervals merge enough.

Brute Force

Check all pairs.

Optimal Approach

Priority queue or counter.

Key Observation

Sort events; active count.

Complexity

Time: `O(n log n)` · Space: `O(n)`

Algorithm

- Events +1/-1
- Sort
- Sweep

Java Template

```
int[][] events; Arrays.sort(events,...)
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Sweep Line Easy** (Easy): Canonical easy problem — Priority queue or counter.

Medium Examples

- **Sweep Line Medium** (Medium): Interview variant — Priority queue or counter.

Hard / Interview Examples

- **Sweep Line Hard** (Hard): Google-style twist — Priority queue or counter.

30-Second Recognition: Triggers: skyline, meeting rooms II

> Trap: Using Sweep Line when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#62 ADVANCED Mastered (+10 XP)

Prefix + Hashing Combo

Why does Prefix + Hashing Combo work? Prefix sum with mod/count map.

Mental Model: Subarray sum k classic.

Recognition Signals: "subarray sum k", "count subarrays"

When to use: Exact subarray count/sum.

When NOT to use: Non-contiguous.

Brute Force

All pairs.

Optimal Approach

Map prefix mod counts.

Key Observation

prefix[j]-prefix[i]=k.

Complexity

Time: O(n) · Space: O(n)

Algorithm

- prefix
- map
- update count

Java Template

```
Map<Long,Integer> cnt=new HashMap<>();
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Prefix + Hashing Combo Easy** (Easy): Canonical easy problem — Map prefix mod counts.

Medium Examples

- **Prefix + Hashing Combo Medium** (Medium): Interview variant — Map prefix mod counts.

Hard / Interview Examples

- **Prefix + Hashing Combo Hard** (Hard): Google-style twist — Map prefix mod counts.

30-Second Recognition: Triggers: subarray sum k, count subarrays

> Trap: Using Prefix + Hashing Combo when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#63 ADVANCED Mastered (+10 XP)

Binary Search + Greedy

Why does Binary Search + Greedy work? Check feasibility greedily for BS answer.

Mental Model: Split array largest sum.

Recognition Signals: "minimize max", "capacity to ship"

When to use: Partition with greedy check.

When NOT to use: No monotonic check.

Brute Force

Try all partitions.

Optimal Approach

BS on answer + greedy.

Key Observation

Greedy pack into minimal bins.

Complexity

Time: $O(n \log S)$ · Space: $O(1)$

Algorithm

- BS lo,hi
- can(mid) greedy

Java Template

```
BS + greedy pack
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Binary Search + Greedy Easy** (Easy): Canonical easy problem — BS on answer + greedy.

Medium Examples

- **Binary Search + Greedy Medium** (Medium): Interview variant — BS on answer + greedy.

Hard / Interview Examples

- **Binary Search + Greedy Hard** (Hard): Google-style twist — BS on answer + greedy.

30-Second Recognition: Triggers: minimize max, capacity to ship

> Trap: Using Binary Search + Greedy when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

#64 ADVANCED Mastered (+10 XP)

Graph + DP Combinations

Why does Graph + DP Combinations work? DAG DP longest path; tree DP on graphs.

Mental Model: Topological order then relax.

Recognition Signals: "longest path DAG", "critical path"

When to use: DP on directed acyclic graph.

When NOT to use: Cycles — not DAG.

Brute Force

All paths.

Optimal Approach

Topo + relax edges.

Key Observation

Topo order enables 1D DP on nodes.

Complexity

Time: O(V+E) · Space: O(V)

Algorithm

- Topo sort
- dp[node]
- Relax

Java Template

```
Topo + dp longest
```

Common Mistakes

- Not handling edge cases
- Wrong complexity analysis

Edge Cases

- Empty input
- Single element
- Maximum constraints

Easy Examples

- **Graph + DP Combinations Easy** (Easy): Canonical easy problem — Topo + relax edges.

Medium Examples

- **Graph + DP Combinations Medium** (Medium): Interview variant — Topo + relax edges.

Hard / Interview Examples

- **Graph + DP Combinations Hard** (Hard): Google-style twist — Topo + relax edges.

30-Second Recognition: Triggers: longest path DAG, critical path

> Trap: Using Graph + DP Combinations when constraints don't fit.

> Interview: Follow-up: How does complexity change with constraints? · Variation: Combine with another pattern for harder variants.

Java Interview Templates

All DSA implementations use Java — your interview language.

HashMap Frequency

```
Map<Integer, Integer> freq = new HashMap<>();
for (int x : nums) {
    freq.merge(x, 1, Integer::sum);
}
```

HashSet

```
Set<Integer> seen = new HashSet<>();
for (int x : nums) {
    if (!seen.add(x)) { /* duplicate */ }
}
```

ArrayList

```
List<Integer> list = new ArrayList<>();
list.add(x);
// list.get(i), list.size()
```

PriorityQueue (Min-Heap)

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
// Max-heap:
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
// Custom comparator:
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
```

Deque

```
Deque<Integer> dq = new ArrayDeque<>();
dq.offerLast(x);
dq.pollFirst();
dq.peekLast();
```

Stack via Deque

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(x);  // offerFirst
int top = stack.pop();  // pollFirst
```

Binary Search

```
int lo = 0, hi = n - 1;
while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;
    if (nums[mid] == target) return mid;
    if (nums[mid] < target) lo = mid + 1;
    else hi = mid - 1;
}
return -1;
```

BFS

```
Queue<int[]> q = new ArrayDeque<>();
boolean[][] vis = new boolean[rows][cols];
q.offer(new int[]{sr, sc});
vis[sr][sc] = true;
int[][] dirs = {{0,1},{1,0},{0,-1},{-1,0}};
while (!q.isEmpty()) {
    int[] cur = q.poll();
    for (int[] d : dirs) {
        int nr = cur[0] + d[0], nc = cur[1] + d[1];
        if (nr < 0 || nc < 0 || nr >= rows || nc >= cols || vis[nr][nc]) continue;
        vis[nr][nc] = true;
        q.offer(new int[]{nr, nc});
    }
}
```

DFS (Recursive)

```
void dfs(int u, List<List<Integer>> adj, boolean[] vis) {
    vis[u] = true;
    for (int v : adj.get(u)) {
        if (!vis[v]) dfs(v, adj, vis);
    }
}
```

TreeNode

```
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int v) { val = v; }
}
```

ListNode

```
class ListNode {
    int val;
    ListNode next;
    ListNode(int v) { val = v; }
}
```

Graph Adjacency List

```
int n = nodes;
List<List<Integer>> adj = new ArrayList<>();
for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
for (int[] e : edges) {
    adj.get(e[0]).add(e[1]);
    adj.get(e[1]).add(e[0]); // undirected
}
```

Union Find

```
class UnionFind {
    int[] parent, rank;
    UnionFind(int n) {
        parent = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }
    boolean union(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;
        if (rank[ra] < rank[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra;
        if (rank[ra] == rank[rb]) rank[ra]++;
        return true;
    }
}
```

Dijkstra

```
long[] dist = new long[n];
Arrays.fill(dist, Long.MAX_VALUE);
dist[src] = 0;
PriorityQueue<long[]> pq = new PriorityQueue<>((a, b) -> Long.compare(a[0], b[0]));
pq.offer(new long[]{0, src});
while (!pq.isEmpty()) {
    long[] cur = pq.poll();
    long d = cur[0];
    int u = (int) cur[1];
    if (d > dist[u]) continue;
    for (int[] e : adj.get(u)) {
        int v = e[0];
        long nd = d + e[1];
        if (nd < dist[v]) {
            dist[v] = nd;
            pq.offer(new long[]{nd, v});
        }
    }
}
```

Topological Sort (Kahn)

```
int[] indeg = new int[n];
for (int[] e : edges) indeg[e[1]]++;
Queue<Integer> q = new ArrayDeque<>();
for (int i = 0; i < n; i++) if (indeg[i] == 0) q.offer(i);
List<Integer> order = new ArrayList<>();
while (!q.isEmpty()) {
    int u = q.poll();
    order.add(u);
    for (int v : adj.get(u)) {
        if (--indeg[v] == 0) q.offer(v);
    }
}
```

Trie

```
class TrieNode {
    TrieNode[] children = new TrieNode[26];
    boolean isWord;
}
class Trie {
    TrieNode root = new TrieNode();
    void insert(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            int i = c - 'a';
            if (node.children[i] == null) node.children[i] = new TrieNode();
            node = node.children[i];
        }
        node.isWord = true;
    }
}
```

Backtracking

```
List<List<Integer>> res = new ArrayList<>();
List<Integer> path = new ArrayList<>();
void backtrack(int idx) {
    if (/* goal */) { res.add(new ArrayList<>(path)); return; }
    for (int i = idx; i < n; i++) {
        path.add(nums[i]);
        backtrack(i + 1);
        path.remove(path.size() - 1);
    }
}
```

DP Memoization

```
Map<String, Integer> memo = new HashMap<>();
int solve(int i, int j) {
    String key = i + "," + j;
    if (memo.containsKey(key)) return memo.get(key);
    int ans = /* recurse */;
    memo.put(key, ans);
    return ans;
}
```

DP Tabulation

```
int[] dp = new int[n + 1];
dp[0] = base;
for (int i = 1; i <= n; i++) {
    dp[i] = Math.max(dp[i - 1], dp[i - 2] + nums[i]);
}
return dp[n];
```

Java-Specific Pitfalls

Pitfall	Fix
Integer overflow	Use long for sums/products; mid = lo + (hi-lo)/2
Comparator	PriorityQueue needs consistent comparator; avoid overflow in (a-b)
PriorityQueue behavior	peek() doesn't remove; poll() removes; default is min-heap
StringBuilder	Use for string concatenation in loops, not +=
Arrays.sort	Primitives: Arrays.sort; objects need Comparator
HashMap frequency	freq.merge(k,1,Integer::sum) or getOrDefault
Recursion depth	Java ~1000-10000; deep trees need iterative
Mutable objects	Don't put mutable lists in HashSet keys
equals/hashCode	Custom objects in HashMap need both
Primitive vs wrapper	int[] faster; Integer in collections

> Trap: Using (a[0]-b[0]) comparator when values can overflow — use Integer.compare or Long.compare.

> In Google interviews, state time/space complexity after coding and mention edge cases aloud.

Low-Level Design (LLD)

SOLID Principles

Principle	Explanation
SRP	Single Responsibility: One class, one reason to change — UserService shouldn't send emails AND validate AND persist
OCP	Open/Closed: Open for extension, closed for modification — Add new PaymentMethod without changing PaymentProcessor
LSP	Liskov Substitution: Subtypes must be substitutable — Square shouldn't break Rectangle setWidth/setHeight contract
ISP	Interface Segregation: No fat interfaces — Split Readable/Writable instead of one huge Repository
DIP	Dependency Inversion: Depend on abstractions — Inject PaymentGateway interface, not StripeClient

Core OOP Concepts

- Composition over inheritance — favor has-a over is-a
- Interfaces for contracts; abstract classes for shared state
- Encapsulation — private fields, public behavior
- Dependency Injection — constructor injection preferred
- Immutability — final fields, unmodifiable collections where possible

Design Patterns

Strategy

Problem: Swap algorithms at runtime

Bad design: Giant if-else or god class

Better: PricingStrategy: Regular, Premium, Discount

Interview Q: When would you NOT use Strategy?

Factory

Problem: Create objects without specifying class

Bad design: Giant if-else or god class

Better: VehicleFactory.create(type)

Interview Q: When would you NOT use Factory?

Abstract Factory

Problem: Families of related objects

Bad design: Giant if-else or god class

Better: UIFactory: WinButton + WinCheckbox

Interview Q: When would you NOT use Abstract Factory?

Builder

Problem: Step-by-step complex construction

Bad design: Giant if-else or god class

Better: HttpRequest.Builder().url().header().build()

Interview Q: When would you NOT use Builder?

Observer

Problem: Publish-subscribe

Bad design: Giant if-else or god class

Better: OrderService notifies Inventory + Notification

Interview Q: When would you NOT use Observer?

Decorator

Problem: Add behavior without subclassing

Bad design: Giant if-else or god class

Better: BufferedInputStream wraps FileInputStream

Interview Q: When would you NOT use Decorator?

Adapter

Problem: Convert interface

Bad design: Giant if-else or god class

Better: LegacyPaymentAdapter implements PaymentGateway

Interview Q: When would you NOT use Adapter?

Facade

Problem: Simplified API over subsystem

Bad design: Giant if-else or god class

Better: OrderFacade hides inventory + payment + shipping

Interview Q: When would you NOT use Facade?

Command

Problem: Encapsulate request as object

Bad design: Giant if-else or god class

Better: Undo/redo with Command stack

Interview Q: When would you NOT use Command?

State

Problem: Behavior changes with state

Bad design: Giant if-else or god class

Better: VendingMachine: Idle, HasMoney, Dispensing

Interview Q: When would you NOT use State?

Singleton

Problem: One instance — often overused

Bad design: Giant if-else or god class

Better: Prefer DI container scoped bean; hard to test

Interview Q: When would you NOT use Singleton?

LLD Problems — Full Framework

Parking Lot

Requirements → Entities

Clarify: actors, core use cases, scale (single machine vs distributed), concurrency needs.

Key Entities & Interfaces

- Define nouns: Vehicle, Spot, Ticket, Payment (example for Parking Lot)
- Interfaces for extensibility: PricingStrategy, Notifier, Storage

Design Patterns

Strategy (pricing), Factory (vehicle types), Observer (notifications), Singleton only if justified.

Concurrency

synchronized / ReentrantLock on shared spot map; consider ConcurrentHashMap for spot registry.

Java Skeleton

```
// Parking Lot — interview skeleton
interface Repository<T, ID> {
    Optional<T> findById(ID id);
    void save(T entity);
}

enum Status { ACTIVE, INACTIVE }

class Service {
    private final Repository<?, ?> repo;
    Service(Repository<?, ?> repo) { this.repo = repo; }
    // core methods
}
```

Extensibility

New vehicle types, pricing rules, or payment methods via new implementations — not modifying core.

Interview Follow-up

How would you scale to multiple floors/buildings? How handle concurrent booking conflicts?

Elevator

Requirements → Entities

Clarify: actors, core use cases, scale (single machine vs distributed), concurrency needs.

Key Entities & Interfaces

- Define nouns: Vehicle, Spot, Ticket, Payment (example for Parking Lot)
- Interfaces for extensibility: PricingStrategy, Notifier, Storage

Design Patterns

Strategy (pricing), Factory (vehicle types), Observer (notifications), Singleton only if justified.

Concurrency

synchronized / ReentrantLock on shared spot map; consider ConcurrentHashMap for spot registry.

Java Skeleton

```
// Elevator — interview skeleton
interface Repository<T, ID> {
    Optional<T> findById(ID id);
    void save(T entity);
}

enum Status { ACTIVE, INACTIVE }

class Service {
    private final Repository<?, ?> repo;
    Service(Repository<?, ?> repo) { this.repo = repo; }
    // core methods
}
```

Extensibility

New vehicle types, pricing rules, or payment methods via new implementations — not modifying core.

Interview Follow-up

How would you scale to multiple floors/buildings? How handle concurrent booking conflicts?

Library Management

Requirements → Entities

Clarify: actors, core use cases, scale (single machine vs distributed), concurrency needs.

Key Entities & Interfaces

- Define nouns: Vehicle, Spot, Ticket, Payment (example for Parking Lot)
- Interfaces for extensibility: PricingStrategy, Notifier, Storage

Design Patterns

Strategy (pricing), Factory (vehicle types), Observer (notifications), Singleton only if justified.

Concurrency

synchronized / ReentrantLock on shared spot map; consider ConcurrentHashMap for spot registry.

Java Skeleton

```
// Library Management — interview skeleton
interface Repository<T, ID> {
    Optional<T> findById(ID id);
    void save(T entity);
}

enum Status { ACTIVE, INACTIVE }

class Service {
    private final Repository<?, ?> repo;
    Service(Repository<?, ?> repo) { this.repo = repo; }
    // core methods
}
```

Extensibility

New vehicle types, pricing rules, or payment methods via new implementations — not modifying core.

Interview Follow-up

How would you scale to multiple floors/buildings? How handle concurrent booking conflicts?

Vending Machine

Requirements → Entities

Clarify: actors, core use cases, scale (single machine vs distributed), concurrency needs.

Key Entities & Interfaces

- Define nouns: Vehicle, Spot, Ticket, Payment (example for Parking Lot)
- Interfaces for extensibility: PricingStrategy, Notifier, Storage

Design Patterns

Strategy (pricing), Factory (vehicle types), Observer (notifications), Singleton only if justified.

Concurrency

synchronized / ReentrantLock on shared spot map; consider ConcurrentHashMap for spot registry.

Java Skeleton

```
// Vending Machine — interview skeleton
interface Repository<T, ID> {
    Optional<T> findById(ID id);
    void save(T entity);
}

enum Status { ACTIVE, INACTIVE }

class Service {
    private final Repository<?, ?> repo;
    Service(Repository<?, ?> repo) { this.repo = repo; }
    // core methods
}
```

Extensibility

New vehicle types, pricing rules, or payment methods via new implementations — not modifying core.

Interview Follow-up

How would you scale to multiple floors/buildings? How handle concurrent booking conflicts?

Splitwise

Requirements → Entities

Clarify: actors, core use cases, scale (single machine vs distributed), concurrency needs.

Key Entities & Interfaces

- Define nouns: Vehicle, Spot, Ticket, Payment (example for Parking Lot)
- Interfaces for extensibility: PricingStrategy, Notifier, Storage

Design Patterns

Strategy (pricing), Factory (vehicle types), Observer (notifications), Singleton only if justified.

Concurrency

synchronized / ReentrantLock on shared spot map; consider ConcurrentHashMap for spot registry.

Java Skeleton

```
// Splitwise — interview skeleton
interface Repository<T, ID> {
    Optional<T> findById(ID id);
    void save(T entity);
}

enum Status { ACTIVE, INACTIVE }

class Service {
    private final Repository<?, ?> repo;
    Service(Repository<?, ?> repo) { this.repo = repo; }
    // core methods
}
```

Extensibility

New vehicle types, pricing rules, or payment methods via new implementations — not modifying core.

Interview Follow-up

How would you scale to multiple floors/buildings? How handle concurrent booking conflicts?

Tic-Tac-Toe

Requirements → Entities

Clarify: actors, core use cases, scale (single machine vs distributed), concurrency needs.

Key Entities & Interfaces

- Define nouns: Vehicle, Spot, Ticket, Payment (example for Parking Lot)
- Interfaces for extensibility: PricingStrategy, Notifier, Storage

Design Patterns

Strategy (pricing), Factory (vehicle types), Observer (notifications), Singleton only if justified.

Concurrency

synchronized / ReentrantLock on shared spot map; consider ConcurrentHashMap for spot registry.

Java Skeleton

```
// Tic-Tac-Toe — interview skeleton
interface Repository<T, ID> {
    Optional<T> findById(ID id);
    void save(T entity);
}

enum Status { ACTIVE, INACTIVE }

class Service {
    private final Repository<?, ?> repo;
    Service(Repository<?, ?> repo) { this.repo = repo; }
    // core methods
}
```

Extensibility

New vehicle types, pricing rules, or payment methods via new implementations — not modifying core.

Interview Follow-up

How would you scale to multiple floors/buildings? How handle concurrent booking conflicts?

Chess

Requirements → Entities

Clarify: actors, core use cases, scale (single machine vs distributed), concurrency needs.

Key Entities & Interfaces

- Define nouns: Vehicle, Spot, Ticket, Payment (example for Parking Lot)
- Interfaces for extensibility: PricingStrategy, Notifier, Storage

Design Patterns

Strategy (pricing), Factory (vehicle types), Observer (notifications), Singleton only if justified.

Concurrency

synchronized / ReentrantLock on shared spot map; consider ConcurrentHashMap for spot registry.

Java Skeleton

```
// Chess — interview skeleton
interface Repository<T, ID> {
    Optional<T> findById(ID id);
    void save(T entity);
}

enum Status { ACTIVE, INACTIVE }

class Service {
    private final Repository<?, ?> repo;
    Service(Repository<?, ?> repo) { this.repo = repo; }
    // core methods
}
```

Extensibility

New vehicle types, pricing rules, or payment methods via new implementations — not modifying core.

Interview Follow-up

How would you scale to multiple floors/buildings? How handle concurrent booking conflicts?

Cab Booking

Requirements → Entities

Clarify: actors, core use cases, scale (single machine vs distributed), concurrency needs.

Key Entities & Interfaces

- Define nouns: Vehicle, Spot, Ticket, Payment (example for Parking Lot)
- Interfaces for extensibility: PricingStrategy, Notifier, Storage

Design Patterns

Strategy (pricing), Factory (vehicle types), Observer (notifications), Singleton only if justified.

Concurrency

synchronized / ReentrantLock on shared spot map; consider ConcurrentHashMap for spot registry.

Java Skeleton

```
// Cab Booking — interview skeleton
interface Repository<T, ID> {
    Optional<T> findById(ID id);
    void save(T entity);
}

enum Status { ACTIVE, INACTIVE }

class Service {
    private final Repository<?, ?> repo;
    Service(Repository<?, ?> repo) { this.repo = repo; }
    // core methods
}
```

Extensibility

New vehicle types, pricing rules, or payment methods via new implementations — not modifying core.

Interview Follow-up

How would you scale to multiple floors/buildings? How handle concurrent booking conflicts?

Logger

Requirements → Entities

Clarify: actors, core use cases, scale (single machine vs distributed), concurrency needs.

Key Entities & Interfaces

- Define nouns: Vehicle, Spot, Ticket, Payment (example for Parking Lot)
- Interfaces for extensibility: PricingStrategy, Notifier, Storage

Design Patterns

Strategy (pricing), Factory (vehicle types), Observer (notifications), Singleton only if justified.

Concurrency

synchronized / ReentrantLock on shared spot map; consider ConcurrentHashMap for spot registry.

Java Skeleton

```
// Logger — interview skeleton
interface Repository<T, ID> {
    Optional<T> findById(ID id);
    void save(T entity);
}

enum Status { ACTIVE, INACTIVE }

class Service {
    private final Repository<?, ?> repo;
    Service(Repository<?, ?> repo) { this.repo = repo; }
    // core methods
}
```

Extensibility

New vehicle types, pricing rules, or payment methods via new implementations — not modifying core.

Interview Follow-up

How would you scale to multiple floors/buildings? How handle concurrent booking conflicts?

Rate Limiter

Requirements → Entities

Clarify: actors, core use cases, scale (single machine vs distributed), concurrency needs.

Key Entities & Interfaces

- Define nouns: Vehicle, Spot, Ticket, Payment (example for Parking Lot)
- Interfaces for extensibility: PricingStrategy, Notifier, Storage

Design Patterns

Strategy (pricing), Factory (vehicle types), Observer (notifications), Singleton only if justified.

Concurrency

synchronized / ReentrantLock on shared spot map; consider ConcurrentHashMap for spot registry.

Java Skeleton

```
// Rate Limiter — interview skeleton
interface Repository<T, ID> {
    Optional<T> findById(ID id);
    void save(T entity);
}

enum Status { ACTIVE, INACTIVE }

class Service {
    private final Repository<?, ?> repo;
    Service(Repository<?, ?> repo) { this.repo = repo; }
    // core methods
}
```

Extensibility

New vehicle types, pricing rules, or payment methods via new implementations — not modifying core.

Interview Follow-up

How would you scale to multiple floors/buildings? How handle concurrent booking conflicts?

Google L4 LLD: clear class diagram, SOLID, extensibility, basic concurrency. L5: deeper trade-offs, multi-tenant, observability.

High-Level Design (HLD / System Design)

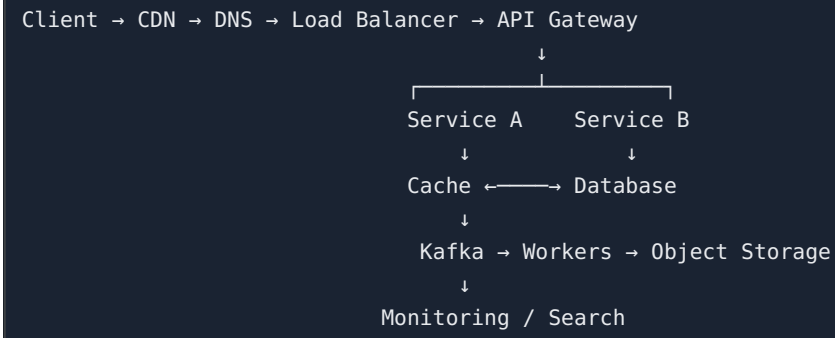
Why does Kafka exist if REST APIs already exist? REST is sync request-response; Kafka is durable async log for decoupling, replay, and burst absorption.

HLD Framework — 28 Concepts

Concept	Why / What problem
Requirement clarification	Scope the problem before designing
Functional requirements	What the system must do
Non-functional requirements	Scale, latency, availability, consistency
QPS estimation	DAU × actions/day ÷ 86400
Storage estimation	Records × size × retention
API design	REST/gRPC endpoints, idempotency keys
Caching	Cache-aside, write-through, TTL strategy
Load balancing	L4 vs L7 — your resume: Dell L4/L7 experience
Kafka	Async decoupling, replay, ordering per partition
Replication	Leader-follower, read replicas
Sharding	Partition by user_id hash
Consistency	Strong vs eventual — CAP trade-off
Idempotency	Safe retries with idempotency keys
Rate limiting	Token bucket, sliding window
Circuit breaker	Fail fast when downstream unhealthy
Backpressure	Slow consumers don't crash producers
Observability	Metrics, logs, distributed tracing

Architecture Building Blocks

Component	Role / Failure / Scaling
Client	User-facing app | Without it: No entry point | Scale: CDN cache static assets
CDN	Edge cache for static content | Without it: High latency, origin overload | Scale: Geographic distribution
DNS	Resolve domain to IP | Without it: Users can't find service | Scale: GeoDNS routing
Load Balancer	Distribute traffic L4/L7 | Without it: Single server bottleneck | Scale: Round-robin, least conn, health checks
API Gateway	Auth, rate limit, routing | Without it: Services exposed directly | Scale: Per-route policies
Service	Business logic microservice | Without it: No functionality | Scale: Horizontal scale, stateless
Cache	Redis/Memcached hot data | Without it: DB overload | Scale: TTL, eviction, cache-aside
Database	Persistent storage | Without it: No durability | Scale: Replication, sharding
Kafka	Durable event streaming | Without it: Tight coupling, lost events | Scale: Partitions, consumer groups
Object Storage	S3 blobs/media | Without it: DB bloat | Scale: Cheap durable storage
Search	Elasticsearch full-text | Without it: Slow SQL LIKE queries | Scale: Inverted index
Monitoring	Metrics, logs, traces | Without it: Blind to failures | Scale: Prometheus, Grafana, alerting



System Design Questions (15)

URL Shortener L4

Requirements

Clarify functional + NFR: scale, latency p99, availability SLA, consistency needs.

Capacity Estimation

QPS, storage/year, bandwidth. Show math on whiteboard.

API

Core endpoints with request/response. Idempotency for writes.

High-Level Architecture

Client → CDN → LB → API Gateway → Services → Cache/DB/Kafka

Data Model

Key tables/documents, indexes, shard key.

Scaling

Horizontal service scale, DB sharding, read replicas, async via Kafka.

Failure Handling

Retries with backoff, circuit breaker, dead letter queue, failover.

L4 Follow-up

How cache invalidation works? What happens when DB primary fails?

L5 Follow-up

Multi-region consistency? Cost optimization? Hot key problem?

☐ Completed URL Shortener design (+50 XP)

Rate Limiter L4

Requirements

Clarify functional + NFR: scale, latency p99, availability SLA, consistency needs.

Capacity Estimation

QPS, storage/year, bandwidth. Show math on whiteboard.

API

Core endpoints with request/response. Idempotency for writes.

High-Level Architecture

Client → CDN → LB → API Gateway → Services → Cache/DB/Kafka

Data Model

Key tables/documents, indexes, shard key.

Scaling

Horizontal service scale, DB sharding, read replicas, async via Kafka.

Failure Handling

Retries with backoff, circuit breaker, dead letter queue, failover.

L4 Follow-up

How cache invalidation works? What happens when DB primary fails?

L5 Follow-up

Multi-region consistency? Cost optimization? Hot key problem?

☐ Completed Rate Limiter design (+50 XP)

Notification System L4

Requirements

Clarify functional + NFR: scale, latency p99, availability SLA, consistency needs.

Capacity Estimation

QPS, storage/year, bandwidth. Show math on whiteboard.

API

Core endpoints with request/response. Idempotency for writes.

High-Level Architecture

Client → CDN → LB → API Gateway → Services → Cache/DB/Kafka

Data Model

Key tables/documents, indexes, shard key.

Scaling

Horizontal service scale, DB sharding, read replicas, async via Kafka.

Failure Handling

Retries with backoff, circuit breaker, dead letter queue, failover.

L4 Follow-up

How cache invalidation works? What happens when DB primary fails?

L5 Follow-up

Multi-region consistency? Cost optimization? Hot key problem?

☐ Completed Notification System design (+50 XP)

File Storage L4

Requirements

Clarify functional + NFR: scale, latency p99, availability SLA, consistency needs.

Capacity Estimation

QPS, storage/year, bandwidth. Show math on whiteboard.

API

Core endpoints with request/response. Idempotency for writes.

High-Level Architecture

Client → CDN → LB → API Gateway → Services → Cache/DB/Kafka

Data Model

Key tables/documents, indexes, shard key.

Scaling

Horizontal service scale, DB sharding, read replicas, async via Kafka.

Failure Handling

Retries with backoff, circuit breaker, dead letter queue, failover.

L4 Follow-up

How cache invalidation works? What happens when DB primary fails?

L5 Follow-up

Multi-region consistency? Cost optimization? Hot key problem?

☐ Completed File Storage design (+50 XP)

Chat System L4

Requirements

Clarify functional + NFR: scale, latency p99, availability SLA, consistency needs.

Capacity Estimation

QPS, storage/year, bandwidth. Show math on whiteboard.

API

Core endpoints with request/response. Idempotency for writes.

High-Level Architecture

Client → CDN → LB → API Gateway → Services → Cache/DB/Kafka

Data Model

Key tables/documents, indexes, shard key.

Scaling

Horizontal service scale, DB sharding, read replicas, async via Kafka.

Failure Handling

Retries with backoff, circuit breaker, dead letter queue, failover.

L4 Follow-up

How cache invalidation works? What happens when DB primary fails?

L5 Follow-up

Multi-region consistency? Cost optimization? Hot key problem?

☐ Completed Chat System design (+50 XP)

YouTube L4

Requirements

Clarify functional + NFR: scale, latency p99, availability SLA, consistency needs.

Capacity Estimation

QPS, storage/year, bandwidth. Show math on whiteboard.

API

Core endpoints with request/response. Idempotency for writes.

High-Level Architecture

Client → CDN → LB → API Gateway → Services → Cache/DB/Kafka

Data Model

Key tables/documents, indexes, shard key.

Scaling

Horizontal service scale, DB sharding, read replicas, async via Kafka.

Failure Handling

Retries with backoff, circuit breaker, dead letter queue, failover.

L4 Follow-up

How cache invalidation works? What happens when DB primary fails?

L5 Follow-up

Multi-region consistency? Cost optimization? Hot key problem?

☐ Completed YouTube design (+50 XP)

Instagram L4

Requirements

Clarify functional + NFR: scale, latency p99, availability SLA, consistency needs.

Capacity Estimation

QPS, storage/year, bandwidth. Show math on whiteboard.

API

Core endpoints with request/response. Idempotency for writes.

High-Level Architecture

Client → CDN → LB → API Gateway → Services → Cache/DB/Kafka

Data Model

Key tables/documents, indexes, shard key.

Scaling

Horizontal service scale, DB sharding, read replicas, async via Kafka.

Failure Handling

Retries with backoff, circuit breaker, dead letter queue, failover.

L4 Follow-up

How cache invalidation works? What happens when DB primary fails?

L5 Follow-up

Multi-region consistency? Cost optimization? Hot key problem?

☐ Completed Instagram design (+50 XP)

Uber L4

Requirements

Clarify functional + NFR: scale, latency p99, availability SLA, consistency needs.

Capacity Estimation

QPS, storage/year, bandwidth. Show math on whiteboard.

API

Core endpoints with request/response. Idempotency for writes.

High-Level Architecture

Client → CDN → LB → API Gateway → Services → Cache/DB/Kafka

Data Model

Key tables/documents, indexes, shard key.

Scaling

Horizontal service scale, DB sharding, read replicas, async via Kafka.

Failure Handling

Retries with backoff, circuit breaker, dead letter queue, failover.

L4 Follow-up

How cache invalidation works? What happens when DB primary fails?

L5 Follow-up

Multi-region consistency? Cost optimization? Hot key problem?

☐ Completed Uber design (+50 XP)

Food Delivery L4/L5

Requirements

Clarify functional + NFR: scale, latency p99, availability SLA, consistency needs.

Capacity Estimation

QPS, storage/year, bandwidth. Show math on whiteboard.

API

Core endpoints with request/response. Idempotency for writes.

High-Level Architecture

Client → CDN → LB → API Gateway → Services → Cache/DB/Kafka

Data Model

Key tables/documents, indexes, shard key.

Scaling

Horizontal service scale, DB sharding, read replicas, async via Kafka.

Failure Handling

Retries with backoff, circuit breaker, dead letter queue, failover.

L4 Follow-up

How cache invalidation works? What happens when DB primary fails?

L5 Follow-up

Multi-region consistency? Cost optimization? Hot key problem?

☐ Completed Food Delivery design (+50 XP)

Ticket Booking L4/L5

Requirements

Clarify functional + NFR: scale, latency p99, availability SLA, consistency needs.

Capacity Estimation

QPS, storage/year, bandwidth. Show math on whiteboard.

API

Core endpoints with request/response. Idempotency for writes.

High-Level Architecture

Client → CDN → LB → API Gateway → Services → Cache/DB/Kafka

Data Model

Key tables/documents, indexes, shard key.

Scaling

Horizontal service scale, DB sharding, read replicas, async via Kafka.

Failure Handling

Retries with backoff, circuit breaker, dead letter queue, failover.

L4 Follow-up

How cache invalidation works? What happens when DB primary fails?

L5 Follow-up

Multi-region consistency? Cost optimization? Hot key problem?

☐ Completed Ticket Booking design (+50 XP)

Payment System L4/L5

Requirements

Clarify functional + NFR: scale, latency p99, availability SLA, consistency needs.

Capacity Estimation

QPS, storage/year, bandwidth. Show math on whiteboard.

API

Core endpoints with request/response. Idempotency for writes.

High-Level Architecture

Client → CDN → LB → API Gateway → Services → Cache/DB/Kafka

Data Model

Key tables/documents, indexes, shard key.

Scaling

Horizontal service scale, DB sharding, read replicas, async via Kafka.

Failure Handling

Retries with backoff, circuit breaker, dead letter queue, failover.

L4 Follow-up

How cache invalidation works? What happens when DB primary fails?

L5 Follow-up

Multi-region consistency? Cost optimization? Hot key problem?

☐ Completed Payment System design (+50 XP)

Distributed Job Scheduler L4/L5

Requirements

Clarify functional + NFR: scale, latency p99, availability SLA, consistency needs.

Capacity Estimation

QPS, storage/year, bandwidth. Show math on whiteboard.

API

Core endpoints with request/response. Idempotency for writes.

High-Level Architecture

Client → CDN → LB → API Gateway → Services → Cache/DB/Kafka

Data Model

Key tables/documents, indexes, shard key.

Scaling

Horizontal service scale, DB sharding, read replicas, async via Kafka.

Failure Handling

Retries with backoff, circuit breaker, dead letter queue, failover.

L4 Follow-up

How cache invalidation works? What happens when DB primary fails?

L5 Follow-up

Multi-region consistency? Cost optimization? Hot key problem?

☐ Completed Distributed Job Scheduler design (+50 XP)

Metrics/Logging Platform L4/L5

Requirements

Clarify functional + NFR: scale, latency p99, availability SLA, consistency needs.

Capacity Estimation

QPS, storage/year, bandwidth. Show math on whiteboard.

API

Core endpoints with request/response. Idempotency for writes.

High-Level Architecture

Client → CDN → LB → API Gateway → Services → Cache/DB/Kafka

Data Model

Key tables/documents, indexes, shard key.

Scaling

Horizontal service scale, DB sharding, read replicas, async via Kafka.

Failure Handling

Retries with backoff, circuit breaker, dead letter queue, failover.

L4 Follow-up

How cache invalidation works? What happens when DB primary fails?

L5 Follow-up

Multi-region consistency? Cost optimization? Hot key problem?

☐ Completed Metrics/Logging Platform design (+50 XP)

Web Crawler L4/L5

Requirements

Clarify functional + NFR: scale, latency p99, availability SLA, consistency needs.

Capacity Estimation

QPS, storage/year, bandwidth. Show math on whiteboard.

API

Core endpoints with request/response. Idempotency for writes.

High-Level Architecture

Client → CDN → LB → API Gateway → Services → Cache/DB/Kafka

Data Model

Key tables/documents, indexes, shard key.

Scaling

Horizontal service scale, DB sharding, read replicas, async via Kafka.

Failure Handling

Retries with backoff, circuit breaker, dead letter queue, failover.

L4 Follow-up

How cache invalidation works? What happens when DB primary fails?

L5 Follow-up

Multi-region consistency? Cost optimization? Hot key problem?

☐ Completed Web Crawler design (+50 XP)

Search Autocomplete L4/L5

Requirements

Clarify functional + NFR: scale, latency p99, availability SLA, consistency needs.

Capacity Estimation

QPS, storage/year, bandwidth. Show math on whiteboard.

API

Core endpoints with request/response. Idempotency for writes.

High-Level Architecture

Client → CDN → LB → API Gateway → Services → Cache/DB/Kafka

Data Model

Key tables/documents, indexes, shard key.

Scaling

Horizontal service scale, DB sharding, read replicas, async via Kafka.

Failure Handling

Retries with backoff, circuit breaker, dead letter queue, failover.

L4 Follow-up

How cache invalidation works? What happens when DB primary fails?

L5 Follow-up

Multi-region consistency? Cost optimization? Hot key problem?

☐ Completed Search Autocomplete design (+50 XP)

L4: end-to-end design with trade-offs. L5: deep dives on consistency, cost, org-scale migration, failure domains.

Resume Deep Dive — Google Questions

Important: Questions below are generated from resume bullets you provided. Do not claim experience you cannot defend. Upload `Pratham_Resume.pdf` for bullet-level customization.

Resume Experience Map

Topic	Context
Dell Technologies	Employer context for backend/distributed systems experience
Layer 4/Layer 7 load balancing	L4 packet-level vs L7 HTTP routing — health checks, SSL termination
S3/REST API traffic	Object storage integration, REST API design for cloud storage
Active storage nodes	Storage cluster with active node management
Multi-Agent Systems	LLM agent orchestration architecture
LLM/RAG	Retrieval-augmented generation pipelines
Private cloud	On-prem / private cloud deployment constraints
Cloud storage APIs	API layer for storage operations
Cluster scaling	Horizontal scaling of storage/compute clusters
Health checks	LB and service health probe patterns
Multi-site VDC	Virtual datacenter across sites
Geographic replication	Cross-region data replication
Global failover	DR and failover orchestration
Spring Boot	Java microservices framework
Kafka	Async messaging, event-driven architecture
Microservices	Service decomposition and communication
Asynchronous Java	CompletableFuture, reactive patterns
Performance optimization	Profiling, bottleneck identification
AI migration agent	AI-assisted migration tooling

Questions Google Can Ask From Your Resume

Layer 4/Layer 7 load balancing

Context: L4 packet-level vs L7 HTTP routing — health checks, SSL termination

Type	Question
Basic	Explain Layer 4/Layer 7 load balancing and where you used it.
Deep Technical	Walk through the implementation details of Layer 4/Layer 7 load balancing in your project.
Architecture	How did Layer 4/Layer 7 load balancing fit into the overall system architecture?
Scaling	What bottlenecks did you hit with Layer 4/Layer 7 load balancing and how did you scale?
Failure	What happens when Layer 4/Layer 7 load balancing fails? How did you detect and recover?
Trade-off	What alternatives to Layer 4/Layer 7 load balancing did you consider and why did you choose this?
L5 Follow-up	How would you redesign Layer 4/Layer 7 load balancing for 10x scale or multi-region?

Kafka

Context: Async messaging, event-driven architecture

Type	Question
Basic	Explain Kafka and where you used it.
Deep Technical	Walk through the implementation details of Kafka in your project.
Architecture	How did Kafka fit into the overall system architecture?
Scaling	What bottlenecks did you hit with Kafka and how did you scale?
Failure	What happens when Kafka fails? How did you detect and recover?
Trade-off	What alternatives to Kafka did you consider and why did you choose this?
L5 Follow-up	How would you redesign Kafka for 10x scale or multi-region?

Geographic replication

Context: Cross-region data replication

Type	Question
Basic	Explain Geographic replication and where you used it.
Deep Technical	Walk through the implementation details of Geographic replication in your project.
Architecture	How did Geographic replication fit into the overall system architecture?
Scaling	What bottlenecks did you hit with Geographic replication and how did you scale?
Failure	What happens when Geographic replication fails? How did you detect and recover?
Trade-off	What alternatives to Geographic replication did you consider and why did you choose this?
L5 Follow-up	How would you redesign Geographic replication for 10x scale or multi-region?

Global failover

Context: DR and failover orchestration

Type	Question
Basic	Explain Global failover and where you used it.
Deep Technical	Walk through the implementation details of Global failover in your project.
Architecture	How did Global failover fit into the overall system architecture?
Scaling	What bottlenecks did you hit with Global failover and how did you scale?
Failure	What happens when Global failover fails? How did you detect and recover?
Trade-off	What alternatives to Global failover did you consider and why did you choose this?
L5 Follow-up	How would you redesign Global failover for 10x scale or multi-region?

LLM/RAG

Context: Retrieval-augmented generation pipelines

Type	Question
Basic	Explain LLM/RAG and where you used it.
Deep Technical	Walk through the implementation details of LLM/RAG in your project.
Architecture	How did LLM/RAG fit into the overall system architecture?
Scaling	What bottlenecks did you hit with LLM/RAG and how did you scale?
Failure	What happens when LLM/RAG fails? How did you detect and recover?
Trade-off	What alternatives to LLM/RAG did you consider and why did you choose this?
L5 Follow-up	How would you redesign LLM/RAG for 10x scale or multi-region?

Multi-Agent Systems

Context: LLM agent orchestration architecture

Type	Question
Basic	Explain Multi-Agent Systems and where you used it.
Deep Technical	Walk through the implementation details of Multi-Agent Systems in your project.
Architecture	How did Multi-Agent Systems fit into the overall system architecture?
Scaling	What bottlenecks did you hit with Multi-Agent Systems and how did you scale?
Failure	What happens when Multi-Agent Systems fails? How did you detect and recover?
Trade-off	What alternatives to Multi-Agent Systems did you consider and why did you choose this?
L5 Follow-up	How would you redesign Multi-Agent Systems for 10x scale or multi-region?

S3/REST API traffic

Context: Object storage integration, REST API design for cloud storage

Type	Question
Basic	Explain S3/REST API traffic and where you used it.
Deep Technical	Walk through the implementation details of S3/REST API traffic in your project.
Architecture	How did S3/REST API traffic fit into the overall system architecture?
Scaling	What bottlenecks did you hit with S3/REST API traffic and how did you scale?
Failure	What happens when S3/REST API traffic fails? How did you detect and recover?
Trade-off	What alternatives to S3/REST API traffic did you consider and why did you choose this?
L5 Follow-up	How would you redesign S3/REST API traffic for 10x scale or multi-region?

Cluster scaling

Context: Horizontal scaling of storage/compute clusters

Type	Question
Basic	Explain Cluster scaling and where you used it.
Deep Technical	Walk through the implementation details of Cluster scaling in your project.
Architecture	How did Cluster scaling fit into the overall system architecture?
Scaling	What bottlenecks did you hit with Cluster scaling and how did you scale?
Failure	What happens when Cluster scaling fails? How did you detect and recover?
Trade-off	What alternatives to Cluster scaling did you consider and why did you choose this?
L5 Follow-up	How would you redesign Cluster scaling for 10x scale or multi-region?

Spring Boot microservices

Context: From resume

Type	Question
Basic	Explain Spring Boot microservices and where you used it.
Deep Technical	Walk through the implementation details of Spring Boot microservices in your project.
Architecture	How did Spring Boot microservices fit into the overall system architecture?
Scaling	What bottlenecks did you hit with Spring Boot microservices and how did you scale?
Failure	What happens when Spring Boot microservices fails? How did you detect and recover?
Trade-off	What alternatives to Spring Boot microservices did you consider and why did you choose this?
L5 Follow-up	How would you redesign Spring Boot microservices for 10x scale or multi-region?

AI migration agent

Context: AI-assisted migration tooling

Type	Question
Basic	Explain AI migration agent and where you used it.
Deep Technical	Walk through the implementation details of AI migration agent in your project.
Architecture	How did AI migration agent fit into the overall system architecture?
Scaling	What bottlenecks did you hit with AI migration agent and how did you scale?
Failure	What happens when AI migration agent fails? How did you detect and recover?
Trade-off	What alternatives to AI migration agent did you consider and why did you choose this?
L5 Follow-up	How would you redesign AI migration agent for 10x scale or multi-region?

Performance optimization

Context: Profiling, bottleneck identification

Type	Question
Basic	Explain Performance optimization and where you used it.
Deep Technical	Walk through the implementation details of Performance optimization in your project.
Architecture	How did Performance optimization fit into the overall system architecture?
Scaling	What bottlenecks did you hit with Performance optimization and how did you scale?
Failure	What happens when Performance optimization fails? How did you detect and recover?
Trade-off	What alternatives to Performance optimization did you consider and why did you choose this?
L5 Follow-up	How would you redesign Performance optimization for 10x scale or multi-region?

Dangerous Claims — Defend or Soften

Claim	What you MUST be able to explain
Layer 4/Layer 7 load balancing	Must explain OSI layers, health check types, sticky sessions, SSL termination
Geographic replication	Must explain sync vs async replication, RPO/RTO, conflict resolution
Global failover	Must explain DNS failover, split-brain, quorum, data consistency during failover
Kafka	Must explain partitions, consumer groups, ordering guarantees, exactly-once semantics
Multi-Agent Systems / LLM/ RAG	Must explain retrieval pipeline, chunking, embedding, latency, hallucination mitigation
Performance optimization	Must cite specific metrics before/after, profiling tools, bottleneck identified
AI migration agent	Must explain agent workflow, human-in-loop, validation, rollback strategy

Trap: Saying 'I built Kafka' when you consumed/produced events — be precise about your role.

Use STAR format: Situation, Task, Action, Result — with metrics where possible.

7-Day Ultra-Intensive Plan

8–10 focused hours/day. Goal: complete mental map, not 300 problems.

DAY 1: Arrays + Hashing + Two Pointers + Sliding Window

Patterns: 1-10 · **Problems:** 12

Learning Blocks

- [ ] 2h Pattern Cards 1-10
- [ ] 2h Recognition drills
- [ ] 2h Solve 6 mediums
- [ ] 1h Active recall quiz
- [ ] 1h Revision + EOD test

Problem Categories

- Frequency Map
- Prefix Sum
- Two Pointers
- Sliding Window fixed/variable
- Kadane
- Intervals

Active Recall

- When HashMap vs Prefix Sum?
- Fixed vs variable sliding window?
- Two pointer on unsorted?

30-Min Revision

Re-read pattern cards for today. Close laptop. Recite triggers from memory.

End-of-Day Test

5 recognition drills + 2 problems timed (45 min each for Day 7 mock).

Pass Criteria

Recognize 8/10 patterns in 30s; solve 4/6 mediums with optimal complexity

Daily reflection: What was hardest? What pattern did I miss?

- [ ] Day 1 Complete (+100 XP Boss Battle)

Quiz: Day 1: Name 3 pattern triggers from today's topics. Reveal Answer

DAY 2: Binary Search + Linked List + Stack + Queue + Heap

Patterns: 11-30 · **Problems:** 12

Learning Blocks

- [ ] 2h Patterns 11-30 cards
- [ ] 2h Java templates practice
- [ ] 2h 6 problems
- [ ] 1h Monotonic stack drills
- [ ] 1h EOD test

Problem Categories

- Binary Search variants
- Linked List
- Monotonic Stack/Queue
- Heap Top-K
- K-way Merge

Active Recall

- BS on answer vs classic BS?
- When monotonic stack?
- Min-heap for top K largest?

30-Min Revision

Re-read pattern cards for today. Close laptop. Recite triggers from memory.

End-of-Day Test

5 recognition drills + 2 problems timed (45 min each for Day 7 mock).

Pass Criteria

Implement 5 templates from memory; 4/6 problems optimal

Daily reflection: What was hardest? What pattern did I miss?

- [ ] Day 2 Complete (+100 XP Boss Battle)

Quiz: Day 2: Name 3 pattern triggers from today's topics. Reveal Answer

DAY 3: Trees + BST + Recursion + Backtracking

Patterns: 31-49 · **Problems:** 10

Learning Blocks

- [ ] 2h Tree patterns
- [ ] 2h Backtracking framework
- [ ] 2h 5 tree + 3 backtrack problems
- [ ] 1h Recursion → iterative
- [ ] 1h EOD test

Problem Categories

- DFS/BFS tree
- LCA
- BST
- Backtracking subsets/perms

Active Recall

- BST validation approach?
- Backtrack template?
- When tree DP?

30-Min Revision

Re-read pattern cards for today. Close laptop. Recite triggers from memory.

End-of-Day Test

5 recognition drills + 2 problems timed (45 min each for Day 7 mock).

Pass Criteria

Solve LCA + one backtracking medium; explain recursion stack depth

Daily reflection: What was hardest? What pattern did I miss?

- [ ] Day 3 Complete (+100 XP Boss Battle)

Quiz: Day 3: Name 3 pattern triggers from today's topics. Reveal Answer

DAY 4: Graphs + BFS + DFS + Topo Sort + DSU + Shortest Path

Patterns: 38-45 · **Problems:** 10

Learning Blocks

- [ ] 2h Graph patterns
- [ ] 2h Template coding
- [ ] 2h 6 graph problems
- [ ] 1h Dijkstra vs BFS
- [ ] 1h EOD test

Problem Categories

- Grid BFS
- Topo sort
- Union Find
- Dijkstra
- Cycle detection

Active Recall

- BFS vs Dijkstra?
- When Union Find?
- Detect cycle directed?

30-Min Revision

Re-read pattern cards for today. Close laptop. Recite triggers from memory.

End-of-Day Test

5 recognition drills + 2 problems timed (45 min each for Day 7 mock).

Pass Criteria

Implement Dijkstra + Topo from memory; 4/6 graph problems

Daily reflection: What was hardest? What pattern did I miss?

- [ ] Day 4 Complete (+100 XP Boss Battle)

Quiz: Day 4: Name 3 pattern triggers from today's topics. Reveal Answer

DAY 5: Greedy + DP + Trie + Bit Manipulation + Advanced

Patterns: 50-64 · **Problems:** 10

Learning Blocks

- [ ] 2h DP state definition drills
- [ ] 2h Advanced patterns
- [ ] 2h 6 DP/Greedy problems
- [ ] 1h Bit manipulation
- [ ] 1h EOD test

Problem Categories

- Greedy proof
- 1D/2D DP
- Knapsack
- Trie
- Bit tricks
- Combo patterns

Active Recall

- Define DP state before code?
- Greedy vs DP decision?
- When trie?

30-Min Revision

Re-read pattern cards for today. Close laptop. Recite triggers from memory.

End-of-Day Test

5 recognition drills + 2 problems timed (45 min each for Day 7 mock).

Pass Criteria

Solve 3 DP mediums; explain state transition aloud

Daily reflection: What was hardest? What pattern did I miss?

- [ ] Day 5 Complete (+100 XP Boss Battle)

Quiz: Day 5: Name 3 pattern triggers from today's topics. Reveal Answer

DAY 6: LLD Framework + Design Patterns + 3 Major Problems

Patterns: LLD · **Problems:** 3

Learning Blocks

- [ ] 2h SOLID + patterns
- [ ] 2h Parking Lot full design
- [ ] 2h Elevator + Rate Limiter
- [ ] 1h UML practice
- [ ] 1h Mock LLD

Problem Categories

- SOLID
- 11 design patterns
- Parking Lot
- Elevator
- Rate Limiter

Active Recall

- SRP example?
- Strategy vs Factory?
- Rate limiter algorithms?

30-Min Revision

Re-read pattern cards for today. Close laptop. Recite triggers from memory.

End-of-Day Test

5 recognition drills + 2 problems timed (45 min each for Day 7 mock).

Pass Criteria

Complete Parking Lot class diagram + Java skeleton in 45 min

Daily reflection: What was hardest? What pattern did I miss?

- [ ] Day 6 Complete (+100 XP Boss Battle)

Quiz: Day 6: Name 3 pattern triggers from today's topics. Reveal Answer

DAY 7: HLD Framework + 5 System Designs + Full Mock Interview

Patterns: HLD · **Problems:** 5

Learning Blocks

- [ ] 2h HLD framework review
- [ ] 3h 5 system designs (35 min each)
- [ ] 2h Full mock (DSA + SD)
- [ ] 1h Cheat sheet revision
- [ ] 1h GOOGLE READY checklist

Problem Categories

- URL Shortener
- Chat
- YouTube
- Uber
- Notification System

Active Recall

- Estimate QPS formula?
- Kafka use case?
- Cache invalidation?

30-Min Revision

Re-read pattern cards for today. Close laptop. Recite triggers from memory.

End-of-Day Test

5 recognition drills + 2 problems timed (45 min each for Day 7 mock).

Pass Criteria

Complete 2 system designs end-to-end; mock interview score ≥70%

Daily reflection: What was hardest? What pattern did I miss?

- [ ] Day 7 Complete (+100 XP Boss Battle)

Quiz: Day 7: Name 3 pattern triggers from today's topics. Reveal Answer

Google Interview Simulation Mode

Interview Mode: cover the 'Pattern' and 'Solution' columns. Think aloud first. Reveal hints only when stuck.

DSA Simulation

Problem (Pattern Hidden)

Given an array of integers, return indices of the two numbers that add up to target. Exactly one solution exists.

Think aloud. What do you notice?

Your approach notes...

Hint 1 (click only if stuck)
What information do you need at each index?
Hint 2
Can you avoid checking all pairs?
Hint 3
Store complement in a map as you iterate.
Reveal Pattern
Frequency Map / HashMap
Reveal Solution
O(n) HashMap: for each nums[i], check if target-nums[i] exists in map.

Mark Mock Complete (+100 XP)

Problem (Pattern Hidden)

Given a string, find the length of the longest substring without repeating characters.

Think aloud. What do you notice?

Your approach notes...

Hint 1 (click only if stuck)
Is the answer contiguous?
Hint 2
What makes a window invalid?
Hint 3
Shrink from left when duplicate found.
Reveal Pattern
Sliding Window — Variable
Reveal Solution
Variable sliding window with HashMap char→last index. O(n).
Mark Mock Complete (+100 XP)

Problem (Pattern Hidden)

Design a rate limiter that allows at most N requests per user per minute.

Think aloud. What do you notice?

Your approach notes...

Hint 1 (click only if stuck)
Clarify: per user? distributed?
Hint 2
Token bucket vs sliding window?
Hint 3
Where to store counters?
Reveal Pattern
HLD — Rate Limiter
Reveal Solution
Redis sliding window or token bucket; key=user_id:minute; return 429 when exceeded.
Mark Mock Complete (+100 XP)

System Design Simulation

Design a Chat System (like Google Chat)

Do NOT look at architecture yet. Answer these first:

1. What requirements do you clarify? (1:1, groups, online status, history?)
2. What scale? (DAU, messages/sec)
3. What API endpoints?
4. What storage for messages?
5. How real-time delivery? (WebSocket, push)
6. What fails? How recover?

Your design notes...

Reveal Architecture Hints

WebSocket gateway → Chat Service → Message DB (sharded by conversation_id) → Kafka for async (notifications, search indexing) → Redis for presence/online status.

Mark SD Mock Complete (+100 XP)

Mock Interview Scorecard

Criteria	Score (1-5)
Problem understanding	
Pattern recognition	
Brute force → optimization	
Correctness	
Complexity analysis	
Code quality (Java)	
Communication	

Quiz: After mock: what pattern did you miss most? Reveal Answer

Gamification & Progress

Level: Pattern Explorer XP: 0 / 8000 Streak: track manually

XP Rules

Action	XP
Pattern learned (checkbox)	+10
Problem solved without help	+20
Hard problem solved	+40
Pattern recognized <30s	+25
System design completed	+50
Mock interview completed	+100
Day boss battle complete	+100
Quiz answer revealed (recall)	+5

Level Progression

Level	Title	XP Required	Unlock
1	Pattern Explorer	0	Learn first 15 patterns
2	Pattern Recognizer	500	30s recognition on 30 patterns
3	Problem Solver	1500	Solve 20 mediums without hints
4	Interview Solver	3000	Complete 5 mock DSA rounds
5	System Designer	5000	Complete 8 system designs
6	Google Ready	8000	Pass full mock + checklist 90%

Boss Battles

- Day 1 Boss: 10 pattern recognition in 5 min
- Day 4 Boss: Graph problem + Dijkstra from memory
- Day 6 Boss: Parking Lot LLD in 45 min
- Day 7 Boss: Full mock interview

Weakness Tracker

☐ DP — state definition

☐ Graph — Dijkstra / Topo

☐ Tree — LCA / Tree DP

☐ Heap — K-way merge

☐

System Design — estimation

☐

LLD — concurrency

GOOGLE READY Checklist

☐

Recognize 45+ patterns in 30 seconds

☐

Code BFS, DFS, BS, Heap, UF from memory

☐

Solve 5 DP mediums with clear state definition

☐

Complete 3 LLD designs with SOLID

☐

Complete 5 system designs with estimation

☐

Defend every resume bullet with STAR + metrics

☐

Think aloud clearly under pressure

☐

Score ≥70% on full mock interview

Next 83 Days — Repetition & Depth

After the 7-Day First Pass

Phase	Focus
Days 8-21	Spaced repetition — revisit each pattern category every 3 days
Days 22-45	Problem solving — 3 mediums/day, focus weak patterns
Days 46-60	System design depth — 1 design/day with follow-ups
Days 61-75	Mock interviews — 3x/week full loop
Days 76-90	Resume deep dives + Google-specific prep + rest

- Weekly: 1 full mock interview (DSA + LLD or HLD)
- Daily: 30 min pattern flashcards
- Track weak patterns in Weakness Tracker
- Revisit resume bullets with STAR stories
- Target: 80+ problems with pattern recognition, not 300 blind solves

Final Master Cheat Sheets

DSA Pattern Recognition Map

- Array sorted → BS / Two Ptr
- Contiguous → Window / Prefix
- Count/freq → HashMap
- Top K → Heap
- Next greater → Mono Stack
- Intervals → Sort + Merge / Sweep
- Tree → DFS/BFS/DP
- Graph shortest unweighted → BFS
- Weighted → Dijkstra
- Deps → Topo
- Connectivity → UF
- All combos → Backtrack
- Optimal substructure → DP
- Min-max answer → BS on Answer

Big-O Cheat Sheet

Structure	Access	Search	Insert	Delete
Array	O(1)	O(n)	O(n)	O(n)
HashMap	—	O(1)*	O(1)*	O(1)*
Balanced BST	—	O(log n)	O(log n)	O(log n)
Heap	min O(1)	O(n)	O(log n)	O(log n)

Sort: O(n log n). BFS/DFS: O(V+E). Dijkstra: O((V+E) log V). UF: α(n).

Java DSA Templates

HashMap freq · Deque stack · PriorityQueue heap · BS template · BFS queue · DFS recursive · UF · Dijkstra PQ

See Java Templates section for full code.

LLD Design Principles

- SOLID
- Composition > Inheritance
- Interface segregation
- DI via constructor
- Immutability where possible

Design Patterns Map

Creational: Factory, Builder | Structural: Adapter, Decorator, Facade | Behavioral: Strategy, Observer, Command, State

HLD Building Blocks

CDN → LB → Gateway → Service → Cache/DB/Kafka → Monitoring

System Design Checklist

1. Requirements (functional + NFR)
2. Estimation (QPS, storage)
3. API design
4. Architecture diagram
5. Data model
6. Scaling
7. Caching
8. Async (Kafka)
9. Failure handling
10. Consistency
11. Security
12. Monitoring

Google Interview Checklist

- Clarify input/output/constraints
- Brute force first
- Optimize with pattern
- State complexity
- Code cleanly in Java
- Test edge cases
- Think aloud

What to Say When Stuck

- "Let me clarify the constraints..."
- "Brute force would be $O(n^2)$ by..."
- "The bottleneck is... I'm thinking [pattern] because..."
- "Let me trace through an example..."
- "Edge case: empty input / single element / duplicates"

Last-Day Revision Sheet

1. 30s diagnosis framework
2. Top 10 patterns you miss
3. Java templates on paper
4. 2 system designs sketched
5. Resume STAR stories
6. Sleep well